# Efficient MPC-Based Modulus Conversion for Threshold FHE Decryption

Ivan Damgård, Sebastian Kolby, Claudio Orlandi, Stanislas Pawlak

Aarhus University
{ivan,sk,orlandi,stanislas.pawlak}@cs.au.dk

**Abstract.** We present new techniques for converting secret-shared values between different moduli in arithmetic MPC, without relying on bit decomposition. More concretely, our protocols convert a sharing $[x]_q$ over a source modulus $q$ into a sharing $[x]_t$ over a target modulus $t$, under a mild bound on the size of $x$. We give three variants: a particularly simple protocol for power-of-two moduli, a protocol for arbitrary source modulus and prime target modulus, and a general protocol for arbitrary target modulus via an intermediate prime modulus. All variants use only a constant number of openings and a small amount of preprocessing. We present them in the arithmetic black box model, so they can be instantiated on top of any MPC protocol supporting basic modular arithmetic.
As a main application, we use these techniques to construct efficient threshold decryption protocols for lattice-based fully homomorphic encryption (FHE), including BFV, BGV, and related schemes. The resulting protocols are special-purpose MPC protocols with a small constant number of rounds. They avoid noise flooding, allowing the parameters of the underlying FHE scheme to be chosen without making room for additional decryption noise.
The resulting protocols achieve statistical UC security against malicious adversaries.
We improve substantially on previous work on MPC-based threshold FHE decryption: as a concrete example, the state-of-the-art protocol by Zyskind et al. (ACM CCS 2025) implements decryption of the BFV scheme (with ciphertext modulus $2^{64}$), using about 17.000 bits of preprocessed correlated randomness, while we need only 63.

## 1 Introduction

Lattice-based cryptosystems are central to both post-quantum cryptography and fully homomorphic encryption. However, despite decades of research, caveats remain in the constructions we know. One such problem is that of constructing truly efficient threshold versions of lattice-based cryptosystems, in particular those based on the Learning With Errors (LWE) problem (virtually all well-studied constructions are LWE-based).

In threshold cryptosystems, the secret key is secret-shared among a number of servers, and the goal is to let them collaborate in decrypting a given ciphertext,

such that the only information revealed is the plaintext. As is well-known, this offers a way to improve both security and reliability of public-key systems.

Starting with [BD10], many threshold decryption protocols have been proposed but virtually all of them are based on so-called noise flooding (see the section below on related work for more details). In this approach, each decryption server computes from its share of the secret key and the ciphertext a so-called decryption share. From a large enough set of decryption shares, the plaintext can then be computed, so the decryption protocol can be made non-interactive.

However, each server needs to add noise to its decryption share to make sure it does not reveal information about the secret key. This in turn implies that the parameters of the system must be chosen large enough to "make room" for the noise, as otherwise the decryption would be incorrect. This demand for larger parameters hurts efficiency and makes ciphertexts larger. It also potentially hurts security, as a stronger computational assumption must be made than would otherwise be necessary. Earlier work has tried to circumvent this problem, but so far this has come at the cost of various issues such as weaker security guarantees or large computational overhead (details on this in related work section).

Thus, the obvious question is whether noise flooding can be avoided. In theory, this question is easy to answer. As is well known, using MPC we can compute any function securely. So, one could secret-share the secret key among the servers and then, using any general MPC protocol, compute securely the function that reconstructs the secret key, uses it to decrypt the ciphertext and outputs the resulting plaintext. While this clearly works, using generic MPC this way would be much too inefficient and would require many rounds of interaction.

A natural way around this would be to instead build a special-purpose MPC protocol for decryption. One would still need to avoid noise flooding, hopefully require only a small constant number of rounds, and be efficient in practice. This is unlikely to result in a non-interactive solution, but it offers a very reasonable tradeoff: we get to choose parameters for the system without worrying about the decryption protocol, and pay for this by a small amount of interaction.

We are aware of two recent works following this approach: The solution from Hu et al. [HLW+25] offers a complete set of protocols for key generation and decryption. The decryption protocol works in 2 rounds but suffers from some drawbacks: it is only semi-honest secure and the plaintext is incorrect with some non-zero probability. To make this probability be sufficiently small, the parameters of the system must be chosen sufficiently large compared to the noise in the ciphertext. This is effectively the same problem that occurs with noise flooding, though perhaps less serious, depending on the desired the error probability.

The other work using MPC for decryption is by Zyskind et al. [ZZLP25]. This solution does not have the issue of a decryption error probability that is tied to the parameter choice. Instead they first do a linear computation on the ciphertext involving the secret key, resulting in a secret-shared value $x$ from which the message can be extracted. To do this securely, they use MPC to reduce $x$ modulo the plaintext modulus, resulting in the message in secret-shared form, which can then be opened. For this purpose, they construct a shallow circuit that

performs the modular reduction. Using preprocessing, this circuit is evaluated in 2 rounds, resulting in 3 rounds overall.

**Our Contribution.** In this work, we propose a new suite of UC secure protocols for threshold decryption. As [HLW+25] and [ZZLP25], our decryption protocol is in the preprocessing model. It is statistically and maliciously secure, and depending on the exact instantiations requires between 2 and 4 rounds. It does not suffer from the drawbacks of [HLW+25], and is much simpler than that of [ZZLP25]. In particular, for the most general case, our preprocessed data consists of only 7 secret-shared values, and the on-line phase requires opening of just 3 values and local computation, involving only cheap linear operations. We also present a decryption protocol for the special case handled in [ZZLP25], namely the BFV cryptosystem[1], where all moduli involved are 2-powers. This variant requires 2 preprocessed values, opens 2 values and runs in 2 rounds.

In contrast, in [ZZLP25], one needs to store preprocessed gates for the entire circuit doing the modular reduction, implying that the preprocessed data grows quite quickly (in fact super-polynomially) as parameters increase. We give a more detailed comparison later, but we already mention that for a realistic example choice of parameters, [ZZLP25] requires about 17,000 bits of preprocessed and secret-shared bits per decryption, while we need 63 bits.

At the heart of our contribution is a new method of independent interest for converting a secret-sharing of a value $x$ modulo some number $q$ to a sharing of the same value modulo a different number $t$. The method only requires a mild bound on the size of $x$. In the technical overview, and in the main threshold-decryption applications, we focus on the simple case $x < q/2$. For threshold decryption, this translates to a very mild restriction on the cryptosystem parameters, as detailed below. More generally, our conversion protocol works under the bound $x < (1 - \frac{1}{\beta})q$, for any integer $\beta \geq 2$. Increasing $\beta$ relaxes the bound on $x$, at the cost of the preprocessing growing proportionally with $\beta$, while the online communication remains constant.

We stress that our protocol can be based on any general MPC protocol that can execute basic modular arithmetic. We demonstrate this by formulating our protocols in Arithmetic Black Box (ABB) model. The ABB formalizes what standard MPC protocols can do for us, and this ensures that any secure implementation of the ABB can support our solution.

The preprocessing we need can be done using known results, in particular this covers the case of dishonest majority. We also suggest a new more efficient preprocessing protocol, that works for the settings of malicious adversary, honest majority, and is specialized for a small number of parties. In runs in 3 rounds and requires opening of $\log_2(q)$ values. Finally, we show a general solution that uses the ABB as a black-box. It is less efficient, but demonstrates that the preprocessing can be done, regardless of the ABB implementation we use.

[1] See the technical overview for details on BFV.

### 1.1 Related Work

As discussed, prior work can roughly be divided into two main lines.

The first line, initiated by [BD10], relies on *noise flooding* applied to the linear partial-decryption procedure of the underlying FHE scheme. In this approach, each party computes a decryption share and adds sufficient noise before revealing it, so that the share does not leak information about the secret key. The plaintext can then be reconstructed from the noisy partial decryptions, often with little or no interaction. This is the main appeal of noise-flooding techniques.

Several works have improved this approach over the years [BGG$^+$17, BS23, PS24, OT25], but they still come with important trade-offs. The construction of [BGG$^+$17] obtains one-round threshold decryption with malicious security, but requires flooding with very large noise. Later works [BS23, PS24] reduce the amount of flooding noise, but achieve this by using ad-hoc key-generation procedures based on short secret-sharings: roughly, the secret key is shared in such a way that each party's share remains small. This is a significant restriction on how the threshold key is generated. Moreover, [BS23] does not provide malicious security, while [PS24] obtains malicious security at the cost of two rounds, losing one of the main advantages that motivates noise flooding in the first place, namely non-interactive threshold decryption.

The recent work of [OT25] addresses some of these issues. It uses Shamir secret-sharing of the decryption key and achieves threshold decryption of BFV/BGV ciphertexts in two rounds. In the first round, a server-assisted decryption phase combines ciphertext sanitization and masking techniques. In the second round, each party decrypts its assigned ciphertext share, after which the plaintext can be reconstructed collectively. However, the protocol still does not provide malicious security and remains a two-round protocol. Thus, despite substantial progress, noise-flooding-based protocols are still limited by trade-offs between the number of rounds, key-generation structure, parameter growth, and malicious security.

A second and more recent line of work avoids noise flooding by viewing threshold decryption as a special-purpose MPC task, following the same general approach as earlier work on threshold signatures from generic MPC [DOK$^+$20, BdCE$^+$25]. This changes the trade-off: one gives up non-interactive decryption, but avoids adding decryption noise and can rely on generic MPC techniques for the secret-shared key. In particular, the parameters of the underlying FHE scheme need not be enlarged merely to hide partial decryptions, and the secret key can be stored using any secret-sharing scheme supported by the underlying MPC protocol.

The first protocol in this line is Ajax [HLW$^+$25], which provides MPC-based protocols for threshold key generation and decryption. Its decryption protocol runs in two rounds and uses masking-and-opening techniques to perform the required modulus conversion on the linearly decrypted ciphertext. However, the masking procedure can overflow, so the protocol is not perfectly correct. To make the failure probability negligible, the FHE parameters must be chosen large enough compared to the ciphertext noise. In particular, this may force the ciphertext modulus to grow superpolynomially in the security parameter.

| | BGV-style | | BFV-style | | | |
|---|---|---|---|---|---|---|
| | $t$ prime | any $q, t$ | any $q, t$ | $q, t$ powers of 2 | | |
| Protocol | §3.1 | §3.2 | §3.2 | §3.3 | [ZZLP25] | [BCD$^+$25] |
| Rounds | 3 | 3 | 4 | 2 | 3 | $O(\log k)$ |
| $C_{\text{on}}$ | $2k + 2\ell$ | $4k - 2\ell + 2\sigma$ | $5k - 4\ell + 2\sigma$ | $2k - \ell$ | $2k - \ell + \sqrt{k - \ell}$ | $> 2k + 4 \log k$ |
| $C_{\text{pre}}$ | $2k + 5\ell$ | $8k - 5\ell + 6\sigma$ | $9k - 2\ell + 6\sigma$ | $k - \ell$ | $> k2^{\sqrt{k-\ell}}$ | $6k \log k + 3k$ |

Table 1: Comparison of MPC-based threshold decryption protocols for BGV-style and BFV-style cryptosystems. The ciphertext modulus is $q$, the plaintext modulus is $t$, with $k = \log q$ and $\ell = \log t$. The parameter $\sigma$ is the statistical security parameter. The row $C_{\text{on}}$ gives the online communication in bits per party, and $C_{\text{pre}}$ gives the preprocessing storage in bits per party.

This is undesirable in a plug-and-play threshold decryption setting: even though the protocol avoids explicit noise flooding, the cost is still pushed back into the parameter selection of the underlying FHE scheme.

The work of Zyskind et al. [ZZLP25] removes this correctness issue. Their protocol first performs the linear part of decryption under MPC, obtaining a secret-sharing of an intermediate value $x$, and then securely extracts the plaintext by reducing $x$ modulo the plaintext modulus. The modular reduction is implemented by evaluating a shallow circuit with preprocessing. This gives malicious security and perfect correctness, but the preprocessing contains secret-shared lookup tables. As a result, the preprocessing storage grows superpolynomially in the modulus bitlength.

The work [BCD$^+$25] considers MPC-based protocols for related BFV-style threshold decryption, however with higher round complexity than our protocols.

Table 1 gives a concrete comparison between our protocols and the closest MPC-based approaches. To the best of our knowledge, the prior MPC-based works do not provide directly comparable cost analyses for BGV-style decryption, so the comparison with [ZZLP25] and [BCD$^+$25] focuses on the BFV-style setting. For [ZZLP25], the online costs are close to ours, but we need 2 rounds rather than three, and the preprocessing is much larger: their approach requires preprocessing for lookup tables, whereas our protocols use only a small number of correlated random values. For example, for a 64-bit modulus, the preprocessing drops from about 17,000 bits in [ZZLP25] to 63 bits in our power-of-two protocol. Compared with [BCD$^+$25], our protocol has smaller (constant) round complexity and lower online communication.

### 1.2 Technical overview

**Domain switching.** We start by sketching our method for moving secret-shared data from one domain to another. For generality and simplicity, we will specify it in the Arithmetic Black Box (ABB) model, that is, we assume we are given

an ideal functionality that can do arithmetic operations over various domains. Our method can then be realized using any secure implementation of the ABB.

The notation $[a]_q$ will denote that the ABB holds the value $a$ internally, and that $a$ belongs to $\mathbb{Z}_q$. We will assume that the underlying MPC protocol allows linear computation on secret values by only local operations. This is virtually always the case since protocols are typically based on linear secret-sharing schemes (such as additive secret-sharing, Shamir secret-sharing, replicated secret-sharing, authenticated secret-sharing, etc.). The protocol will then be run by a number of parties who, in this model, can do it by simply issuing commands to the ABB.

The setting is now that we have $[x]_q$ and the goal is to compute $[x]_Q$ where $Q$ is some other value. The only assumption we need is that $x < q/2$, though as mentioned in the introduction, this can be generalized. We will present three protocols: one that works when both $q, Q$ are powers of 2, one that works when $Q$ is prime, and one that works for arbitrary target moduli.

We start by describing our first protocol that only works if $Q$ is prime: From a preprocessing phase, we assume we have two pairs $[r]_q, [r]_Q$ and $[s]_q, [s]_Q$, where $r, s$ are uniformly chosen in $\mathbb{Z}_q$. By linearity, the servers compute and open $x + r \bmod q$ and $2x + s \bmod q$.

Note that since by assumption $x < q/2$, we know that both $x + r$ and $2x + s$ are less than $2q$, so we have, for binary $0/1$ values $c_0, c_1$ that

$$x + r \bmod q = x + r - c_0 q \text{ and } 2x + s \bmod q = 2x + s - c_1 q$$

The servers now input $x + r - c_0 q \bmod Q, 2x + s - c_1 q \bmod Q$ to the ABB and subtract the preprocessed values $r, s$ modulo $Q$, resulting in

$$[x - c_0 q]_Q, [2x - c_1 q]_Q{}^2.$$

By further local computation, the servers multiply the first value by 2 and subtract the last one, so we now have

$$[c_1 q - 2c_0 q]_Q = [q(c_1 - 2c_0)]_Q.$$

The crucial observation now is that $c_1 - 2c_0$ may take 4 different values, that are in 1-1 correspondence with the value of the pair $(c_0, c_1)$. Concretely, the possible values are $0, 1, -2, -1$. Since $Q$ is prime, $q$ is invertible modulo $Q$, so therefore, one can compute $qc_0$ from $q(c_1 - 2c_0)$. Let $f(x)$ be a polynomial over $\mathbb{Z}_Q$ with the property that $f(q(c_1 - 2c_0)) = qc_0$, this is a public polynomial that can be constructed via Lagrange interpolation, to have degree at most 3. We then assume that we have, from preprocessing, $[y]_Q, [y^2]_Q, [y^3]_Q$, where $y$ is uniformly chosen from $\mathbb{Z}_Q$. It is well known that by opening $y + q(c_1 - 2c_0) \bmod Q$, the servers can compute locally $[f(q(c_1 - 2c_0))]_Q = [qc_0]_Q$, as this only requires linear operations.

Finally, the servers add $[qc_0]_Q$ to $[x - qc_0]_Q$ to get $[x]_Q$, as desired. One easily sees that this required just 2 rounds of communication. Moreover, since

[2] What we have is, strictly speaking, $[(x - c_0 q) \bmod Q]_Q$, but we omit the $\bmod Q$ since the context is clear.

the method we have sketched is completely agnostic to the MPC protocol used, we get malicious security by using any maliciously secure MPC protocol, and we can always tolerate as many corruptions as the MPC tolerates.

If the target modulus is not a prime, we may not be able to do the polynomial interpolation needed at the end of the protocol. So, to convert $[x]_q$ to $[x]_t$ for arbitrary $t$, we choose a prime $Q$ such that $Q \gg q$, compute $[x]_Q$ as explained above, and finally use a standard mask-and-open approach to compute $[x]_t$. Namely, we assume a preprocessed pair $[z]_Q$, $[z]_t$, for a random $z$ where $z < Q/2$ but $z \gg x$, so we can securely open $z + x \bmod Q = z + x$, input $z + x \bmod t$ to the ABB and compute $[x]_t$ by local operations.

**Domain switching for powers-of-two.** Some moduli allow for more efficient specialized domain switching protocols due to their structure. One such case is when converting between powers-of-two, a regime relevant to FHE [CGGI20] and well studied within MPC [CDE+18, OSV20]. Once again we present our approach in the ABB model.

We have $[x]_{2^\ell}$ and our goal is to compute $[x]_{2^k}$ where $\ell < k$. As before, we assume $x < 2^{\ell-1}$, or equivalently that the most significant bit of $x$ is 0. Our strategy will still be to mask and open $x$ and then extract the wrap-around bit, but with one key difference: in our preprocessing we sample $r$ uniformly from $\mathbb{Z}_{2^{\ell-1}}$ and *not* the whole range $\mathbb{Z}_{2^\ell}$. We assume $[r]_{2^k}$ and $[r]_{2^\ell}$ from preprocessing, noting that these may be constructed given $\ell-1$ random bits. At this point $x+r$ cannot be opened yet, but we observe

$$x + r \bmod 2^\ell = x + r = y' + c \cdot 2^{\ell-1}$$

where $y' = x + r \bmod 2^{\ell-1}$ and $c \in \{0, 1\}$ as $x + r < 2^\ell$. In fact, $y'$ is uniform in $\mathbb{Z}_{2^{\ell-1}}$, meaning the $\ell - 1$ least significant bits of $x + r$ are already safe to open. The wrap-around bit $c$ is left as the only leakage with respect to $x$ and is necessary if we wish to compute

$$[x]_{2^k} = 2^{\ell-1}[c]_{2^k} + y' - [r]_{2^k}.$$

To obtain $[c]_{2^k}$ we require one additional random bit $b$ preprocessed as $[b]_{2^k}$ and $[b]_{2^\ell}$, allowing us to open $y$,

$$\begin{aligned}[y]_{2^\ell} &= [x]_{2^\ell} + [r]_{2^\ell} + [b]_{2^\ell} \cdot 2^{\ell-1} = [y' + (b + c) \cdot 2^{\ell-1}]_{2^\ell} \\ &= [y' + (b \oplus c) \cdot 2^{\ell-1}]_{2^\ell}.\end{aligned}$$

This is secure as the least significant bits $y'$ were already uniform and we have applied $b$ as a one time pad to $c$. The most significant bit of $y$ tells us whether $b$ is equal to $c$, which we can leverage to get $[c]_{2^k}$ we needed. If the most significant bit of $y$ is 0 then $[c]_{2^k} = [b]_{2^k}$, otherwise $[c]_{2^k} = 1 - [b]_{2^k}$. With this we can compute $[x]_{2^k}$ from $[c]_{2^k}$, $y'$, and $[r]_{2^k}$.

**Threshold decryption.** In this paper, we consider a wide class of cryptosystems, of which BGV [BGV12], BFV [Bra12,FV12] and others are examples. Such cryptosystems make use of two moduli $q$ and $t$, where $q > t$. Moreover, both the

secret key $\mathbf{s}$ and ciphertexts $\mathbf{c}$ are vectors over $\mathbb{Z}_q$, whereas messages $\mathbf{m}$ are in general vectors over $\mathbb{Z}_t$. For simplicity, we will assume for now that the message is a single number $m \in \mathbb{Z}_t$.

The crucial property we need from the cryptosystem is that, given a ciphertext $\mathbf{c}$, there is a linear function $f_{\mathbf{c}}(\cdot)$ such that decryption is a 2-stage process: first one computes $x = f_{\mathbf{c}}(\mathbf{s})$. $f_c$ is usually very simple, often it essentially just computes the inner product of $\mathbf{s}$ and $\mathbf{c}$. In the second stage one extracts the message from $x$. Most known lattice-based cryptosystems work like this.

We now explain how the domain switching technique above can be used to securely decrypt a ciphertext. We assume that $[\mathbf{s}]_q$ has been set up in the ABB during the key generation phase. The decryption algorithm runs in two stages.

In the first stage, given the ciphertext $\mathbf{c}$, the servers compute $[x]_q = [f_c(\mathbf{s})]_q$ using only local computation since $f_c$ is linear.

In the second stage, the message is extracted from $[x]_q$. For BGV-like systems, the message is computed as $m = x \bmod t$. Note, however, that we cannot just open $x$, as it is well known that this would leak information on the noise in $\mathbf{c}$ and hence on the secret key. Instead, we use our domain switching protocol to compute $[x]_t$, which we can securely open, as this only reveals $m = x \bmod t$. This requires 3 rounds in total.

To enable our domain switching technique, we need to assume that the parameters of the cryptosystem are chosen such that $x < q/2$. This is an extremely mild restriction: earlier schemes based on noise flooding required $x$ to be smaller than $q$ by an exponentially large factor. In a nutshell: to use our protocol, you can choose the parameters of the cryptosystem exactly as you like, except that you may need to make $q$ one bit longer.

For BFV-style schemes, the message is extracted differently: here $x = \Delta m + e \bmod q$, where $\Delta = q/t$. We assume here that $t$ divides $q$, which can be done without loss of generality by standard modulus switching. With this assumption, from $[x]_q$ we can get $[x]_\Delta = [e]_\Delta$ using only local operations, as we assume the ABB uses linear secret-sharing: the parties simply reduce their shares modulo $\Delta$. We then use our domain switching technique to compute $[e]_q$, which we can subtract from $[x]_q$ to get $[\Delta m]_q$ which is safe to open.

**Preprocessing.** As mentioned above, the main challenge in preprocessing is to build sharings of the same number in different domains, say $\mathbb{Z}_q, \mathbb{Z}_Q$. This can be done by leveraging existing protocols for creating so-called da-bits. A da-bit is a pair $([b]_q, [b]_2)$, where $b$ is a random bit and $[b]_2$ denotes secret-sharing over the domain $\mathbb{Z}_2$. We can get the required preprocessed data by simple constructions making black-box use of da-bits, which can be generated from previously known protocols. In particular, this covers the dishonest majority setting.

We also propose a more efficient solution tailored for the honest majority setting, a small number of parties, and for $q$ a 2-power. For this, we first observe that it suffices to produce *double-shared bits*: pairs of variables modulo $q$ and $Q$, $[b]_q, [b]_Q$, containing the same bit $b$. Namely, by generating many such pairs we can combine them using linear combinations with 2-powers as coefficients to build numbers of the size we want. Then, we show a protocol for generating

such pairs. It runs in 3 rounds and requires opening only a single value. For this, we use replicated secret-sharing over the integers and exploit the fact that such sharings can be locally converted to sharings over any modulus.

Finally, we show a generic, less efficient protocol for double-shared bits that uses only the standard ABB interface. In this solution, individual parties share the same bit in both domains, prove this was done correctly and then we combine the contributions from the parties.

## 2 Preliminaries and Functionalities

### 2.1 Arithmetic Black Box

We describe our protocols in the arithmetic black box (ABB) model. This allows us to specify the protocol independently of any concrete MPC instantiation.

Informally, an ABB provides an interface for manipulating secret-shared values. The parties interact with the ABB by issuing commands, while the ABB internally stores the corresponding values under identifiers. The parties can perform arithmetic operations on these stored values and can open selected values, but learn nothing else.

We use an ABB supporting modular arithmetic for several moduli, generalizing the functionality presented in [Esc22]. More precisely, for a set of moduli $M$, the functionality $\mathcal{F}_{\mathsf{ABB}}(M)$ supports arithmetic over $\mathbb{Z}_m$ for each $m \in M$. The usual arithmetic commands are input, linear combination, multiplication, and opening. These correspond to standard operations provided by arithmetic MPC protocols.

We also include two preprocessing-oriented commands. The first samples correlated random values over two moduli. This is not a basic arithmetic operation, but it can be implemented using standard preprocessing techniques, for instance from daBits or edaBits [EGK+20]. The second reduces a shared value modulo a divisor of its current modulus, which is a local operation for the usual linear secret-sharing based realizations when the smaller modulus divides the larger one. The formal functionality is given in Figure 1.

**Power-of-two computations.** MPC modulo $2^k$ is a well-studied problem which affords efficient instantiations of some useful additional operations. We will use $\mathcal{F}_{\mathsf{ABB}}(2^k)$ as a shorthand for $\mathcal{F}_{\mathsf{ABB}}(\{2^i\}_{i\in[k]})$. Given an arithmetic black box which only supports arithmetic modulo $2^k$ we may emulate arithmetic for all smaller powers of two. To do so, store $x \in \mathbb{Z}_{2^{m_1}}$ in the high-bits of $y \in \mathbb{Z}_{2^k}$, i.e. $y = x \cdot 2^{k-m_1}$, this encodes $x$ in the sub-ring generated by $2^{k-m_1}$. Conversions to lower powers by the command $\mathsf{divModConv}$ from $\mathbb{Z}_{2^{m_1}}$ to $\mathbb{Z}_{2^{m_2}}$ where $m_2 < m_1$ may simply be performed by multiplying $y$ by the appropriate power-of-two,

$$y' = 2^{m_1-m_2}y = (x \bmod 2^{m_2}) \cdot 2^{k-m_2} \pmod{2^k}.$$

We only add one command specific to the power of two setting: $\mathsf{randBit}$, which allows the ABB to sample and store uniformly random bits. This has various

established instantiations in the literature [DEF+19,EGK+20,RW19]. Given this operation uniformly random values modulo $2^\ell$ can be constructed by a linear combination of $\ell$-bits in the natural way.

**Functionality $\mathcal{F}_{\mathsf{ABB}}(M)$**

$\mathcal{F}_{\mathsf{ABB}}$ is parameterized by a set $M$, and supports arithmetic in $\mathbb{Z}_m$ for every $m \in M$.

**Input:** On input $\mathsf{cmd} = (\mathsf{input}, m, P_i, \mathsf{id})$ from all honest parties, where $m \in M$, send $\mathsf{cmd}$ to the adversary. Wait for input $(\mathsf{input}, m, P_i, \mathsf{id}, x)$ from $P_i$, where $x \in \mathbb{Z}_m$, and store $(\mathsf{id}, m, x)$.

**Linear Combination:** On input

$$\mathsf{cmd} = (\mathsf{comb}, m, \{c_i\}_{i=0}^{\ell}, \{\mathsf{id}_i\}_{i\in[\ell]}, \mathsf{id}_{\ell+1})$$

from all honest parties, retrieve $(\mathsf{id}_i, m, x_i)$ for $i \in [\ell]$, compute

$$z = c_0 + \sum_{i=1}^{\ell} c_i x_i \pmod m,$$

and store $(\mathsf{id}_{\ell+1}, m, z)$. Finally, send $\mathsf{cmd}$ to the adversary.

**Multiplication:** On input $\mathsf{cmd} = (\mathsf{mult}, m, \mathsf{id}_1, \mathsf{id}_2, \mathsf{id}_3)$ from all honest parties, retrieve $(\mathsf{id}_1, m, x)$ and $(\mathsf{id}_2, m, y)$, compute $z = xy \bmod m$, and store $(\mathsf{id}_3, m, z)$. Finally, send $\mathsf{cmd}$ to the adversary.

**Open:** On input $\mathsf{cmd} = (\mathsf{open}, m, \mathsf{id})$ from all honest parties, retrieve $(\mathsf{id}, m, x)$, send $x$ to all parties, and send $\mathsf{cmd}$ to the adversary.

**Divisible Modulus Conversion:** On input

$$\mathsf{cmd} = (\mathsf{divModConv}, m_1, m_2, \mathsf{id}_1, \mathsf{id}_2)$$

from all honest parties, where $m_1, m_2 \in M$ and $m_2$ divides $m_1$, retrieve $(\mathsf{id}_1, m_1, x)$, compute $y = x \bmod m_2$, and store $(\mathsf{id}_2, m_2, y)$. Finally, send $\mathsf{cmd}$ to the adversary.

Prime Moduli

**Double Random:** On input $\mathsf{cmd} = (\mathsf{drand}, m_1, m_2, B, \mathsf{id}_1, \mathsf{id}_2)$ from all honest parties, where $m_1, m_2 \in M$, sample $r \leftarrow \mathbb{Z}_B$, store $(\mathsf{id}_1, m_1, r \bmod m_1)$ and $(\mathsf{id}_2, m_2, r \bmod m_2)$. Finally, send $\mathsf{cmd}$ to the adversary.

Power-of-two Moduli

**Random Bit:** On input $(\mathsf{randBit}, \mathsf{id})$ from all honest parties, sample $b \leftarrow \{0, 1\}$ and store $(\mathsf{id}, b)$, then send $(\mathsf{randBit}, \mathsf{id})$ to the adversary.

Fig. 1: The Multi-Moduli Arithmetic Black Box Functionality $\mathcal{F}_{\mathsf{ABB}}$.

**Conversions.** The main functionality needed in our threshold-decryption protocols, and the main technical contribution of this paper, is a general modulus-conversion command. Given a sharing $[x]_{m_1}$ over a source modulus $m_1$, the command produces a sharing $[x]_{m_2}$ over a target modulus $m_2$, even when the target modulus does not divide the source modulus. Correctness is guaranteed when the integer represented by $x$ lies in a prescribed interval, captured by the parameter $\beta$. We model this command through the ideal functionality $\mathcal{F}_{\mathsf{ABB+Conv}}(M, \beta)$, which extends $\mathcal{F}_{\mathsf{ABB}}(M)$ as shown in Figure 2. Note that this functionality only guarantees privacy and correctness for inputs in the valid interval. If the input is outside this interval, the value $x$ is given to the adversary, who is then allowed to determine the output.

**Identifier namespaces.** In all $\mathcal{F}_{\mathsf{ABB}}$-hybrid protocols, identifiers introduced by the protocol itself are local to that protocol invocation. This includes identifiers for masks, preprocessing values, and intermediate values. It is important that those are not exposed to the outer functionality (and thus the environment or higher-level protocols), otherwise simulation trivially fails. Such identifiers are freshly generated in a namespace disjoint from the identifiers supplied by the calling environment, and cannot be accessed by higher-level protocols unless they are the explicit output of the protocol. Concretely, this can be implemented by prefixing internal identifiers with a fresh session identifier. Thus, when a protocol realizes an extended ABB command, only the input and output identifiers of that command are exposed.

**Functionality $\mathcal{F}_{\mathsf{ABB+Conv}}(M, \beta)$**

$\mathcal{F}_{\mathsf{ABB+Conv}}$ is parameterized by a set of moduli $M$ and a bound parameter $\beta \geq 2$. It provides all commands of $\mathcal{F}_{\mathsf{ABB}}(M)$, and in addition the following command.

**General Modulus Conversion:** On input

$$\mathsf{cmd} = (\mathsf{modConv}, m_1, m_2, \mathsf{id}_1, \mathsf{id}_2)$$

from all honest parties, where $m_1, m_2 \in M$, retrieve $(\mathsf{id}_1, m_1, x)$.

- If $x \geq m_1 - m_1/\beta$, send $(\mathsf{cmd}, x)$ to the adversary, wait for $y \in \mathbb{Z}_{m_2}$, and store $(\mathsf{id}_2, m_2, y)$.
- Otherwise, store $(\mathsf{id}_2, m_2, x \bmod m_2)$ and send $\mathsf{cmd}$ to the adversary.

Fig. 2: Arithmetic Black Box with General Modulus Conversion.

### 2.2 Supported Cryptosystems

We describe an abstract class of lattice-based cryptosystems to which our threshold decryption techniques apply. Such schemes use a ciphertext modulus $q$ and a plaintext modulus $t$, with $q > t$.

We assume that the secret key $\mathbf{s}$ and ciphertexts $\mathbf{c}$ are vectors over $\mathbb{Z}_q$, while plaintexts $\mathbf{m}$ are vectors over $\mathbb{Z}_t$. In many schemes these objects are ring elements or ring vectors rather than plain vectors, but this distinction does not matter for our protocols. The property we need is that decryption proceeds in two stages.

We assume that the first step of decryption applies the secret key to the ciphertext in a linear way. Concretely, the ciphertext $\mathbf{c}$ defines a linear map $f_{\mathbf{c}}$ over $\mathbb{Z}_q$, and decryption first computes

$$\mathbf{x} = f_{\mathbf{c}}(\mathbf{s}).$$

In typical lattice-based schemes, this step is essentially an inner product between the ciphertext and the secret key, or the analogous operation over a structured ring.

The second step extracts the plaintext from $\mathbf{x}$. We consider two common forms of extraction. In BGV-type schemes, the plaintext is obtained directly by reducing the relevant components of $\mathbf{x}$ modulo the plaintext modulus:

$$m = x \bmod t.$$

Looking ahead, threshold decryption thus reduces to converting a sharing of $x$ modulo $q$ into a sharing of the same value modulo $t$, without opening $x$.

In BFV-type schemes, the relevant component has the form

$$x = \Delta m + e \pmod q,$$

where $\Delta$ is a public scaling factor, usually close to $q/t$, and $e$ is a small error term. Here the goal is to remove the error term and then recover $m$ from $\Delta m$. Looking ahead, in our threshold protocol this is done by first reducing $x$ modulo $\Delta$, which yields the error $e$, then converting this sharing back to modulus $q$, subtracting it from $x$, and finally opening $\Delta m$.

We formalize the threshold-decryption task as extensions of the ABB functionality. We do not model key generation here. Instead, we assume that the secret key has already been established inside the ABB under an identifier $\mathsf{id}_{\mathsf{sk}}$, as a sharing modulo $q$. This can be achieved either by a distributed key-generation protocol or by having the parties input their secret-key shares into the ABB. Our decryption protocols only use this ABB handle and are independent of the concrete secret-sharing scheme used to realize it.

The functionalities below differ from standard BGV and BFV decryption only in that they make the required correctness bound explicit through the parameter $\beta$. If the bound is violated, the relevant intermediate value is given to the adversary, who may determine the output. This matches the convention used in $\mathcal{F}_{\mathsf{ABB+Conv}}$. Moreover, for BFV-type schemes, we assume that $\Delta = q/t$ is an integer. This is the usual situation when $q$ and $t$ are powers of two, and otherwise can be arranged by standard modulus switching.

**Functionality $\mathcal{F}_{\mathsf{ABB+BGV}}(M, \beta)$**

$\mathcal{F}_{\mathsf{ABB+BGV}}$ is parameterized by a set of moduli $M$ and a bound parameter $\beta \geq 2$. It provides all commands of $\mathcal{F}_{\mathsf{ABB+Conv}}(M, \beta)$, and in addition the following command.

**BGV Decryption:** On input

$$\mathsf{cmd} = (\mathsf{bgvDec}, q, t, c, \mathsf{id}_{\mathsf{sk}}, \mathsf{id}_{\mathsf{out}})$$

from all honest parties, where $q, t \in M$, $q > t$, and $c = (c_0, c_1)$ is public, retrieve $(\mathsf{id}_{\mathsf{sk}}, q, \mathsf{sk})$ and compute

$$x = f_c(\mathsf{sk}) = c_0 + c_1\mathsf{sk} \bmod q.$$

- If $x \geq q - q/\beta$, send $(\mathsf{cmd}, x)$ to the adversary, wait for $y \in \mathbb{Z}_t$, and store $(\mathsf{id}_{\mathsf{out}}, t, y)$.
- Otherwise, store $(\mathsf{id}_{\mathsf{out}}, t, x \bmod t)$ and send $\mathsf{cmd}$ to the adversary.

Fig. 3: ABB extension for BGV decryption

## 3 Protocols for Modulus Conversion

We present three modulus-conversion protocols. The first handles the case of a prime target modulus. It converts a sharing $[x]_q$ over an arbitrary source modulus $q$ into a sharing $[x]_t$ over a prime target modulus $t$, under the promise that $x < q - q/\beta$, for $\beta \geq 2$.

The second protocol removes the restriction that the target modulus is prime. It uses the first protocol as a subroutine, converting first to a sufficiently large intermediate prime modulus $Q$, and then from $Q$ to the desired target modulus $t$ by a final masking step.

The third protocol is independent of the first two. It is a specialized conversion protocol for power-of-two moduli, and gives a simpler and more efficient solution when both source and target moduli are powers of two.

### 3.1 Prime Modulus Conversion

We first consider the case of a prime target modulus $t$. The goal is to realize $\mathsf{modConv}$ from $[x]_q$ to $[x]_t$, under the promise that $x < q - q/\beta$.

When the target modulus $t$ does not divide $q$, the main difficulty comes from the wrap-around phenomenon occurring during reconstruction. More precisely, reconstructing a shared value modulo $q$ and then interpreting it modulo $t$ may introduce an unknown multiple of $q$, which prevents a direct conversion.

Naively, one might try to compute this wrap-around within MPC in order to correct it. However, there is no simple protocol for this, and the wrap-around may depend on the number of players, making such an approach inefficient.

**Functionality $\mathcal{F}_{\mathsf{ABB+BFV}}(M, \beta)$**

$\mathcal{F}_{\mathsf{ABB+BFV}}$ is parameterized by a set of moduli $M$ and a bound parameter $\beta \geq 2$. It provides all commands of $\mathcal{F}_{\mathsf{ABB+Conv}}(M, \beta)$, and in addition the following command.

**BFV Decryption:** On input

$$\mathsf{cmd} = (\mathsf{bfvDec}, q, t, c, \mathsf{id}_{\mathsf{sk}}, \mathsf{id}_{\mathsf{out}})$$

from all honest parties, where $q, t \in M$, $q > t$, $t$ divides $q$, and $c = (c_0, c_1)$ is public, retrieve $(\mathsf{id}_{\mathsf{sk}}, q, \mathsf{sk})$. Let $\Delta = q/t$, and compute

$$x = f_c(\mathsf{sk}) = \langle c, \mathsf{sk} \rangle \bmod q.$$

Define

$$e = x \bmod \Delta.$$

- If $e \geq \Delta - \Delta/\beta$, send $(\mathsf{cmd}, x)$ to the adversary, wait for $y \in \mathbb{Z}_q$, and store $(\mathsf{id}_{\mathsf{out}}, t, y)$.
- Otherwise, compute

$$m = \frac{(x - e)}{\Delta} \in \mathbb{Z}_t$$

store $(\mathsf{id}_{\mathsf{out}}, q, \Delta m)$, and send $\mathsf{cmd}$ to the adversary.

Fig. 4: ABB extension for BFV decryption.

For example, suppose $x$ is additively shared modulo $q$ among $n$ parties as $x = \sum_{i=1}^{n} x_i \bmod q$. Writing the shares as integers $x_i \in [0, q)$, we have

$$\sum_{i=1}^{n} x_i = x + cq$$

for some $c \in \{0, \ldots, n-1\}$. If the parties simply reinterpret the same shares modulo $t$, they obtain

$$\sum_{i=1}^{n} (x_i \bmod t) = x + cq \pmod t,$$

rather than $x \bmod t$. Thus a direct conversion would require correcting the unknown multiple $cq$, where $c$ depends on the sharing and can grow with the number of parties.

Another idea, exploited in previous work [PS24, BS23], is to use shares of small size to avoid wrap-around altogether. While this is a valid approach, it requires significant overhead or a trusted setup.

Our protocol instead keeps the standard mask-and-open approach, but uses it in a way that avoids the reconstruction-dependent wrap-around described

above. After opening a single masked value modulo $q$, the only remaining wrap-around is the single correction bit indicating whether $x + r$ crossed $q$. We add a small amount of extra information that lets the parties recover this bit. We first explain the idea, and then give the formal protocol and proof.

**High-level idea.** Let $r$ be a uniformly random value, shared both modulo $q$ and modulo $t$. The parties compute and open

$$y = [x]_q + [r]_q \pmod{q}.$$

Since $r$ is uniform modulo $q$, the opened value $y$ reveals no information about $x$. Interpreting $y$ as an integer in $[0, q)$, there exists a bit $c \in \{0, 1\}$ such that

$$y = x + r - cq.$$

Using the sharing of the same mask modulo $t$, the parties can locally compute

$$[X]_t = y - [r]_t = [x - cq]_t.$$

Thus the only remaining task is to remove the correction term $cq$.

**Recovering the wrap-around bit.** One of the main technical ideas in this paper is a novel way for recovering the bit $c$. In particular, we suggest to repeat the above procedure with a second random mask $r'$, but now instead of masking $x$, we mask the multiple $\beta x$ for $\beta \geq 2$, i.e., we open

$$y' = \beta x + r' \bmod q.$$

Using the sharing of the same mask modulo $t$, the parties locally compute

$$[X']_t = y' - [r']_t = [\beta x - c'q]_t,$$

where the corresponding wrap-around value $c'$ is no longer a bit: indeed, $c' \in \{0, \ldots, \beta\}$, since $\beta x < \beta q$.

From $[X]_t = [x - cq]_t$ and $[X']_t = [\beta x - c'q]_t$, the parties compute (here we used that $t$ is prime and $q$ is invertible modulo $t$):

$$[\theta]_t = q^{-1} \cdot \left(\beta[X]_t - [X']_t\right) = [c' - \beta c]_t.$$

Note that the value $c' - \beta c$ almost determines $c$. Indeed, for $(c, c') \in \{0, 1\} \times \{0, \ldots, \beta\}$, the only collision is the value 0, which can arise in two ways:

$$(c, c') = (0, 0) \qquad \text{or} \qquad (c, c') = (1, \beta).$$

Note, however, that the second case only happens if $\beta x + r' \geq \beta q$, or equivalently $x \geq q - r'/\beta$. Hence, if $x < q - q/\beta$, this ambiguity cannot occur for any choice of $r'$.

We can therefore recover $c$ from $\theta = c' - \beta c$ by evaluating a public interpolation polynomial over $\mathbb{Z}_t$. Assuming $t > 2\beta$, let $P_\beta(X)$ be the unique polynomial over $\mathbb{Z}_t$ of degree at most $2\beta - 1$ satisfying

$$P_\beta(z) = \begin{cases} 1 & \text{for } z \in \{-\beta, \ldots, -1\}, \\ 0 & \text{for } z \in \{0, \ldots, \beta - 1\}. \end{cases} \tag{1}$$

Then $P_\beta(\theta) = c$ for all valid inputs. Once $[c]_t$ is obtained, the parties output

$$[x]_t = [X]_t + q[c]_t.$$

**Efficiency.** The protocol uses two calls to $\mathsf{drand}$, producing correlated masks $([r]_q, [r]_t)$ and $([r']_q, [r']_t)$. In addition, the polynomial evaluation of $P_\beta$ uses standard preprocessing for masked polynomial evaluation over $\mathbb{Z}_t$, namely sharings of the powers of a random mask needed to evaluate a degree-$(2\beta-1)$ polynomial.

In the online phase, the parties first open two values modulo $q$, namely $y$ and $y'$. They then compute $[\theta]_t$ locally and evaluate $P_\beta$ on $[\theta]_t$. Using the standard masked polynomial-evaluation subprotocol, this adds one masked opening modulo $t$. Thus the online phase has two rounds: one round for the two openings modulo $q$, and one round for the masked polynomial evaluation over $\mathbb{Z}_t$. In total, the protocol opens two values modulo $q$ and one value modulo $t$ online.

**Protocol** $\mathsf{Prime\text{-}Modulus\text{-}Conversion}(q, t, \beta)$

This protocol is in the $\mathcal{F}_{\mathsf{ABB}}(\{q, t\})$-hybrid model, parameterized by domains $\mathbb{Z}_t$ and $\mathbb{Z}_q$, assuming that $t$ is prime and $q$ is invertible modulo $t$. We denote by $P_\beta(X)$ the polynomial defined in Equation (1).

**Preprocessing:** $[r]_t, [r]_q, [r']_t, [r']_q$, where $(r, r') \leftarrow \mathbb{Z}_q^2$, using two calls to $\mathsf{drand}$.
**I/O:** Given input $[x]_q$ such that $x < q - \frac{q}{\beta}$, produces $[x]_t$ stored in $\mathcal{F}_{\mathsf{ABB}}$.
**Execute:** Each party $P_i$ proceeds as follows:

1. Use $\mathcal{F}_{\mathsf{ABB}}$ to compute:
   - $[y]_q \leftarrow [x]_q + [r]_q$
   - $[y']_q \leftarrow \beta \cdot [x]_q + [r']_q$
2. Call $\mathcal{F}_{\mathsf{ABB}}.\mathsf{Open}([y]_q, [y']_q) \to (y, y')$.
3. Use $\mathcal{F}_{\mathsf{ABB}}$ to compute:
   - $[X]_t \leftarrow (y \bmod t) - [r]_t$
   - $[X']_t \leftarrow (y' \bmod t) - [r']_t$
4. Use $\mathcal{F}_{\mathsf{ABB}}$ to compute
$$[\theta]_t \leftarrow q^{-1}\big(\beta \cdot [X]_t - [X']_t\big).$$
5. Call $\mathcal{F}_{\mathsf{ABB}}.\mathsf{PolyEval}([\theta]_t, P_\beta(X)) \to [c]_t$.
6. Use $\mathcal{F}_{\mathsf{ABB}}$ to compute $[x]_t \leftarrow [X]_t + q[c]_t$.

Fig. 5: Prime target modulus conversion.

**Theorem 1.** *Let $t$ be prime with $q$ invertible modulo $t$, and let $P_\beta$ be defined as in Equation* (1)*. The protocol* $\mathsf{Prime\text{-}Modulus\text{-}Conversion}(t, q, \beta)$ *in Figure 5 realizes the functionality* $\mathcal{F}_{\mathsf{ABB+Conv}}(\{q, t\}, \beta)$ *in the* $\mathcal{F}_{\mathsf{ABB}}(\{q, t\})$*-hybrid model.*

*Proof.* For readability, throughout this proof we write $\mathcal{F}_{\mathsf{ABB}}$ for $\mathcal{F}_{\mathsf{ABB}}(\{q,t\})$.

We define a simulator $\mathcal{S}$ which emulates $\mathcal{F}_{\mathsf{ABB}}$ and the virtual honest parties.

**Simulator $\mathcal{S}$**

- Upon receiving $(\mathsf{input},\cdot)$, $(\mathsf{comb},\cdot)$, $(\mathsf{mult},\cdot)$, $(\mathsf{open},\cdot)$, $(\mathsf{drand},\cdot)$, or $(\mathsf{PolyEval},\cdot)$ from $\mathcal{F}_{\mathsf{ABB}+\mathsf{conv}}$, the simulator emulates $\mathcal{F}_{\mathsf{ABB}}$ calling the functionality on behalf of virtual honest parties. The simulator only needs to know internal values of $\mathcal{F}_{\mathsf{ABB}}$ when they are opened, at which point it receives the necessary values from $\mathcal{F}_{\mathsf{ABB}+\mathsf{conv}}$.
- Upon receiving command $(\mathsf{modConv}, q, t, \mathsf{id}_1, \mathsf{id}_2)$ proceed as follows:
  **Preprocessing:**
  1. Emulate the calls of $\mathsf{drand}$ to $\mathcal{F}_{\mathsf{ABB}}$, resulting in $(r, \mathsf{id}_0')$, $(r', \mathsf{id}_1')$.

  **Execute:**
  1. Emulate the call $\mathsf{comb}$ to $\mathcal{F}_{\mathsf{ABB}}$ for $\mathsf{id}_1, \mathsf{id}_0', \mathsf{id}_1'$, resulting in $(y_0, \mathsf{id}_2')$ and $(y_1, \mathsf{id}_3')$.
  2. Sample random $y_0^*, y_1^* \leftarrow \mathbb{Z}_q^2$ and emulate the commands $(\mathsf{open}, \mathsf{id}_2')$ and $(\mathsf{open}, \mathsf{id}_3')$ by sending $y_0^*$ and $y_1^*$ to the corrupt parties.
  3. Emulate the call $\mathsf{comb}$ to $\mathcal{F}_{\mathsf{ABB}}$ for $\mathsf{id}_2', \mathsf{id}_3'$, resulting in $(\theta, \mathsf{id}_3)$.
  4. Emulate the calls to $\mathcal{F}_{\mathsf{ABB}}$ performed during $\mathsf{PolyEval}$. This includes calls to $\mathsf{comb}$ and one randomly sampled opening.
  5. Emulate the final call $\mathsf{comb}$ to $\mathcal{F}_{\mathsf{ABB}}$, acting as the honest virtual parties.
- Upon receiving command $(\mathsf{modConv}, q, t, \mathsf{id}_1, \mathsf{id}_2, x)$ proceed as follows. (This is the easy case where the simulator runs exactly as in the real world, since it knows the input $x$.)
  1. Run the real protocol and store the resulting output.

We focus only on $(\mathsf{modConv},\cdot)$, as the remaining commands are simply forwarded to $\mathcal{F}_{\mathsf{ABB}}$, which is emulated internally in the simulator. When $x \geq q - \frac{q}{\beta}$, the real and ideal worlds are identical, as the simulator is given $x$ and may proceed exactly as in the real world.

This leaves the case where $x < q - \frac{q}{\beta}$. Here we must argue (1) that the real protocol produces the correct value and (2) that the distribution of the openings in the real and ideal worlds is identical.

(1) By construction, we have

$$y = x + r \bmod q \qquad \text{and} \qquad y' = \beta x + r' \bmod q.$$

Therefore, for some $c \in \{0,1\}$ and $c' \in \{0,\ldots,\beta\}$,

$$y = x + r - cq \qquad \text{and} \qquad y' = \beta x + r' - c'q.$$

After subtracting the masks modulo $t$, the protocol obtains

$$[X]_t = [x - cq]_t \qquad \text{and} \qquad [X']_t = [\beta x - c'q]_t.$$

Thus

$$[\theta]_t = q^{-1}\big(\beta[X]_t - [X']_t\big) = [c' - \beta c]_t.$$

We now check that the polynomial $P_\beta(X)$, defined in Equation (1), recovers the value of $c$ when evaluated on $\theta$. If $c = 0$, then $\theta = c'$. Under the bound $x < q - q/\beta$, the case $c' = \beta$ cannot occur, so $\theta \in \{0, \ldots, \beta-1\}$. Hence $P_\beta(X)(\theta) = 0 = c$.

If $c = 1$, then $\theta = c' - \beta$. For $c' \in \{0, \ldots, \beta - 1\}$, we have $\theta \in \{-\beta, \ldots, -1\}$, and hence $P_\beta(X)(\theta) = 1 = c$. The only remaining case is $c' = \beta$. But this would require $\beta x + r' \geq \beta q$, hence $x \geq q - r'/\beta$. Since $r' < q$, this contradicts $x < q - q/\beta$. Thus $c' = \beta$ cannot occur in the valid interval, and $P_\beta(X)(\theta) = c$ in all cases.

The call to PolyEval therefore returns $[c]_t$. Finally, the protocol adds $q[c]_t$ to $[X]_t = [x - cq]_t$, obtaining

$$[X]_t + q[c]_t = [x - cq]_t + q[c]_t = [x]_t.$$

Therefore the output of the protocol is correct.

(2) Now we argue that the simulated openings have the same distribution as in the real execution. Since $r$ and $r'$ come from calls to drand, they are uniform modulo $q$. Hence $y = x + r \bmod q$ and $y' = \beta x + r' \bmod q$ are uniform in $\mathbb{Z}_q$ and independent of $x$. The simulator can therefore replace the real openings $y, y'$ by independent uniform values $y_0^*, y_1^* \in \mathbb{Z}_q$. The remaining operations are local linear computations, except for the call to PolyEval that consist of another uniformly masked opening. Hence the real and ideal executions are indistinguishable. □

### 3.2 General Modulus Conversion

The protocol of the previous section requires the target modulus to be prime. In some applications, and in particular in the threshold-decryption protocols we consider later, the natural target modulus is instead dictated by the underlying cryptosystem and may not satisfy the constraint.

We remove the restriction by using the prime-modulus protocol as a black-box subroutine. The idea is to first convert from the source modulus $q$ to a sufficiently large intermediate prime modulus $Q$, and then convert from $Q$ down to the desired target modulus $t$ by one final masked opening. Since $Q$ is chosen much larger than $q$, this final step can be done with negligible probability of wrap-around.

**High-level idea.** Let $q$ and $t$ be arbitrary moduli. We choose an intermediate prime modulus $Q$ such that $Q$ is much larger than $q$, say $Q > 2^\kappa q$ for a statistical security parameter $\kappa$.

The first step is to convert the input sharing $[x]_q$ into a sharing $[x]_Q$ by invoking Prime-Modulus-Conversion:

$$[x]_Q \leftarrow \mathsf{Prime\text{-}Modulus\text{-}Conversion}(Q, q, \beta)([x]_q).$$

It remains to convert from the large prime modulus $Q$ to the desired target modulus $t$. For this, let $R$ be a random mask shared both modulo $Q$ and modulo $t$, with $R$ sampled from a range large enough to hide $x$, but small enough that $x + R < Q$ except with negligible probability. The parties open

$$y = [x]_Q + [R]_Q \pmod{Q}$$

and interpret $y$ as an integer. Except with probability at most $2^{-\kappa}$, there is no wrap-around modulo $Q$, so $y = x + R$. The parties can therefore locally compute

$$[x]_t = y - [R]_t.$$

**Efficiency.** The protocol invokes Prime-Modulus-Conversion once, with target modulus $Q$, and then uses one additional correlated mask $([R]_Q, [R]_t)$. The final step opens one value modulo $Q$, namely $y = x + R \bmod Q$, and then computes $[x]_t = y - [R]_t$ locally.

Naively, this adds one correlated mask, one opening modulo $Q$, and one online round compared to the prime-target protocol. However, the final opening can be combined with the last opening used inside Prime-Modulus-Conversion, since the two opened values are independent masked values. With this scheduling, the general protocol has the same number of online rounds as the prime-target protocol, while adding only one extra opened value modulo $Q$ and one extra correlated mask.

**Protocol** General-Modulus-Conversion$(q, t, \beta, \kappa)$

This protocol is in the $\mathcal{F}_{\mathsf{ABB}}(\{q, t, Q\})$-hybrid model, parameterized by source modulus $\mathbb{Z}_q$, target modulus $\mathbb{Z}_t$, and an intermediate prime modulus $\mathbb{Z}_Q$, where $Q$ is chosen so that $Q > 2^\kappa q$.

**Preprocessing:** $[R]_Q, [R]_t$, where $R \leftarrow \mathbb{Z}_Q$, generated using a call to drand.
**I/O:** Given input $[x]_q$ such that $x < q - \frac{q}{\beta}$, produces $[x]_t$ stored in $\mathcal{F}_{\mathsf{ABB}}$.
**Execute:** Each party $P_i$ proceeds as follows:

1. Run Prime-Modulus-Conversion$(q, Q, \beta)$ on input $[x]_q$ to obtain $[x]_Q$.
2. Use $\mathcal{F}_{\mathsf{ABB}}$ to compute $[y]_Q \leftarrow [x]_Q + [R]_Q$.
3. Call $\mathcal{F}_{\mathsf{ABB}}.\mathsf{Open}([y]_Q) \rightarrow y$.
4. Use $\mathcal{F}_{\mathsf{ABB}}$ to compute $[x]_t \leftarrow y - [R]_t$.

Fig. 6: General modulus conversion via an intermediate prime modulus.

**Theorem 2.** *Let $Q$ be a prime modulus such that $Q > 2^\kappa q$. The protocol in Figure 6 General-Modulus-Conversion$(q, t, \beta, \kappa)$ realizes $\mathcal{F}_{ABB+Conv}(\{q, t\}, \beta)$ in the $\mathcal{F}_{ABB}(\{q, t, Q\})$-hybrid model, except with probability at most $2^{-\kappa}$.*

*Proof.* For readability, throughout this proof we write $\mathcal{F}_{\mathsf{ABB}}$ for $\mathcal{F}_{\mathsf{ABB}}(\{q, t, Q\})$.

We define a simulator $\mathcal{S}$ which emulates $\mathcal{F}_{\mathsf{ABB}}$ and the virtual honest parties.

**Simulator $\mathcal{S}$**

- Upon receiving $(\mathsf{input}, \cdot)$, $(\mathsf{comb}, \cdot)$, $(\mathsf{mult}, \cdot)$, $(\mathsf{open}, \cdot)$, or $(\mathsf{drand}, \cdot)$ from $\mathcal{F}_{\mathsf{ABB}+\mathsf{conv}}$, the simulator emulates $\mathcal{F}_{\mathsf{ABB}}$ calling the functionality on behalf of virtual honest parties. The simulator only needs to know internal values of $\mathcal{F}_{\mathsf{ABB}}$ when they are opened, at which point it receives the necessary values from $\mathcal{F}_{\mathsf{ABB}+\mathsf{conv}}$.
- Upon receiving command $(\mathsf{modConv}, q, t, \mathsf{id}_1, \mathsf{id}_2)$ proceed as follows:
  1. Emulate the call to $\mathsf{Prime\text{-}Modulus\text{-}Conversion}(q, Q, \beta)$ on input $(\mathsf{id}_1, \mathsf{id}'_0)$, relying on the simulator from the proof of Theorem 1. This stores the converted value under $(\mathsf{id}'_0, Q, x)$.
  2. Emulate the call of $\mathsf{drand}$ to $\mathcal{F}_{\mathsf{ABB}}$, resulting in correlated masks $[R]_Q$ and $[R]_t$, stored under identifiers $\mathsf{id}'_1$ and $\mathsf{id}'_2$.
  3. Emulate the call $\mathsf{comb}$ to $\mathcal{F}_{\mathsf{ABB}}$ for $\mathsf{id}'_0, \mathsf{id}'_1$, resulting in $(y, \mathsf{id}'_3)$.
  4. Sample a random $y^* \leftarrow \mathbb{Z}_Q$ and emulate the command $(\mathsf{open}, \mathsf{id}'_3)$ by sending $y^*$ to the corrupt parties.
  5. Emulate the final call $\mathsf{comb}$ to $\mathcal{F}_{\mathsf{ABB}}$, acting as the honest virtual parties.
- Upon receiving command $(\mathsf{modConv}, q, t, \mathsf{id}_1, \mathsf{id}_2, x)$ proceed as follows. (This is the easy case where the simulator runs exactly as in the real world, since it knows the input $x$.)
  1. Run the real protocol and store the resulting output.

We focus only on $(\mathsf{modConv}, \cdot)$, as the remaining commands are simply forwarded to $\mathcal{F}_{\mathsf{ABB}}$, which is emulated internally in the simulator. When $x \geq q - \frac{q}{\beta}$, the real and ideal worlds are identical, as the simulator is given $x$ and may proceed exactly as in the real world.

This leaves the case where $x < q - \frac{q}{\beta}$. We must argue (1) that the real protocol produces the correct value except with probability at most $2^{-\kappa}$, and (2) that the distribution of the opening $y$ and the simulated opening $y^*$ are identical in the ideal and simulated worlds.

(1) The call to $\mathsf{Prime\text{-}Modulus\text{-}Conversion}$ gives us the shared input in the intermediate domain $\mathbb{Z}_Q$. We then mask it by the uniform value $R$ and open

$$y = x + R \bmod Q.$$

Assuming that no overflow occurs during this operation, we have $y = x + R$ as integers. The next operation removes the mask by subtracting the shared mask in $\mathbb{Z}_t$, resulting in a sharing of $x$ in $\mathbb{Z}_t$.

An overflow can occur and would lead to an incorrect reconstruction. We therefore do not achieve perfect correctness in this final step, but we can bound

the probability of overflow. Since $x < q$ and $R$ is uniform in $\mathbb{Z}_Q$,

$$\Pr[\text{overflow}] = \Pr[R \geq Q - x] = \frac{x}{Q} < \frac{q}{Q} < 2^{-\kappa},$$

where the last inequality follows from the choice $Q > 2^{\kappa} q$. Therefore, for a correctly chosen intermediate modulus $Q$, we achieve correctness except with probability at most $2^{-\kappa}$.

(2) Now we open $y = x + R \bmod Q$, with $R$ coming from the call to $\mathsf{drand}$. Since $R$ is uniform in $\mathbb{Z}_Q$, the value $y$ is uniform in $\mathbb{Z}_Q$ and independent of $x$. This is identical to the simulated opening $y^* \leftarrow \mathbb{Z}_Q$. Therefore the ideal and real worlds are indistinguishable for the adversary.

□

### 3.3 Power-of-Two Modulus Conversion

Thus far we have dealt with modulus conversion, first converting from prime moduli and then generalizing to arbitrary starting moduli. Our protocols have all been generic, only assuming an appropriate realization of $\mathcal{F}_{\mathsf{ABB}}$.

However, for performance reasons, practical FHE implementations often favor moduli with particular structure that enables more efficient computation. Broadly these moduli may be divided into two categories, CRT structure where $q = p_1 \cdots p_k$ is the product of many smaller primes and prime-power structure where $q = p^k$. In this section we will propose a modulus conversion protocol specific to the latter category, focusing on the specific case $q = 2^k$. Power-of-two moduli are particularly favored in instantiations of Torus FHE [CGGI20] including previous thresholdization approaches [BCD+25].

**High-level Idea.** We wish to convert $[x]_{2^\ell}$ where $x < 2^{\ell-1}$ to $[x]_{2^k}$ where $\ell < k$. Conceptually our approach for the power-of-two case is much the same as before. Mask and open $x$, then extract the bit indicating whether the masking caused a wrap-around. The main difference is that we may exploit zero-divisors of $\mathbb{Z}_{2^k}$.

First, sample a random mask $r \leftarrow [0, 2^{\ell-1})$ which we assume is stored both as $[r]_{2^\ell}$ and $[r]_{2^k}$. Then, add this to $[x]_{2^\ell}$, and assuming $x < 2^{\ell-1}$ this cannot cause an overflow, since $x + r < 2^\ell$. At this point we cannot yet open $[x + r]_{2^\ell}$ as this does not hide $x$. But we may observe that the distribution of $y' = x + r \mod 2^{\ell-1}$ is independent of $x$, meaning that the only leakage is from the most significant bit of $x + r$. Let $c \in \{0, 1\}$ be the bit such that $x + r = y' + 2^{\ell-1} c \bmod 2^\ell$. We may eliminate this leakage using a random bit $b \leftarrow \{0, 1\}$ (also stored as $[b]_{2^\ell}$ and $[b]_{2^k}$). More precisely we compute and open,

$$y \leftarrow [x + r + 2^{\ell-1} b]_{2^\ell} = [y' + 2^{\ell-1}(b + c)]_{2^\ell} = [y' + 2^{\ell-1}(b \oplus c)]_{2^\ell}.$$

This hides $c$, using $b$ as a one-time-pad, and conveniently also lets us obtain $[c]_{2^k} = 1 - [b]_{2^k}$ if the most significant bit of $y$ is 1 and $[c]_{2^k} = [b]_{2^k}$ if it is 0. This allows us to obtain $[x]_{2^k} = y' + 2^{\ell-1}[c]_{2^k} - [r]_{2^k}$ as desired.

We present our formal protocol in Figure 7.

**Protocol** Po2-Modulus-Conversion$(2^\ell, 2^k)$

This protocol is in the $\mathcal{F}_{\mathsf{ABB}}$ $(2^k)$ hybrid model, parameterized by domains $\mathbb{Z}_{2^\ell}$ and $\mathbb{Z}_{2^k}$ assuming $k > \ell$.

**Preprocessing:** $[r]_{2^\ell}, [r]_{2^k}$ where $r \leftarrow [0, 2^{\ell-1})$, and $[b]_{2^\ell}, [b]_{2^k}$ where $b \leftarrow \{0, 1\}$. These values may be produced using the random bit functionality.
**I/O:** Given input $[x]_{2^\ell}$ such that $x < 2^{\ell-1}$ produces $[x]_{2^k}$ stored in $\mathcal{F}_{\mathsf{ABB}}$.
**Execute:** Each party $P_i$ proceeds as follows:

1. Use $\mathcal{F}_{\mathsf{ABB}}$ to compute $[y]_{2^\ell} = [x]_{2^\ell} + [r]_{2^\ell} + 2^{\ell-1}[b]_{2^\ell}$.
2. Call $\mathcal{F}_{\mathsf{ABB}}.\mathsf{Open}([y]_{2^\ell}) \to y$.
3. Let $b'$ be the most significant bit of $y$ and $y' = y \bmod 2^{\ell-1}$, then case on $b' \in \{0, 1\}$:
   - $b' = 0$ then let $[x]_{2^k} \leftarrow y' + 2^{\ell-1}[b]_{2^k} - [r]_{2^k}$
   - $b' = 1$ then let $[x]_{2^k} \leftarrow y' + 2^{\ell-1}(1 - [b]_{2^k}) - [r]_{2^k}$

Fig. 7: Power-of-two modulus conversion.

**Theorem 3.** *The protocol Po2-Modulus-Conversion$(2^\ell, 2^k)$ (Figure 7) realizes the functionality $\mathcal{F}_{ABB+\mathsf{conv}}(2^k, 2)$ (Figure 2) in the $\mathcal{F}_{ABB}(2^k)$-hybrid model.*

*Proof.* For readability, throughout this proof we write $\mathcal{F}_{\mathsf{ABB}}$ for $\mathcal{F}_{\mathsf{ABB}}(2^k)$. We define a simulator in Figure 8.

We focus only on $(\mathsf{modConv}, \cdot)$ as the remaining commands are simply forwarded to $\mathcal{F}_{\mathsf{ABB}}$ emulated internally in the simulator. When $x \geq 2^{\ell-1}$ the real and ideal worlds are identical as the simulator given $x$ may proceed exactly as in the real world.

This leaves the case where $x < 2^{\ell-1}$, here we must argue (1) that the real protocol produces the correct value and (2) that the distribution of the opening $y$ is identical in the real and ideal worlds. (1) By construction we have $x + r < 2^\ell$. This may be decomposed into $y' \in [0, 2^{\ell-1})$ and $c \in \{0, 1\}$ such that

$$x + r = (x + r \bmod 2^\ell) = y' + c \cdot 2^{\ell-1}.$$

Recall,

$$\begin{aligned} y &= x + r + b \cdot 2^{\ell-1} = y' + (b + c) \cdot 2^{\ell-1} \\ &= y' + (b \oplus c) \cdot 2^{\ell-1} = y' + b' \cdot 2^{\ell-1} \pmod{2^\ell} \end{aligned}$$

As $c = b' \oplus b$,

$$\begin{aligned} x &= y' - r + 2^{\ell-1}c = y' - r + 2^{\ell-1}(b' \oplus b) \\ &= y' - r + 2^{\ell-1}(b'(1 - b) + (1 - b')b) \end{aligned}$$

**Simulator $\mathcal{S}$**

We define a simulator $\mathcal{S}$ which emulates $\mathcal{F}_{\mathsf{ABB}}$ and the virtual honest parties.

- Upon receiving $(\mathsf{input}, \cdot)$, $(\mathsf{comb}, \cdot)$, $(\mathsf{mult}, \cdot)$,$(\mathsf{open}, \cdot)$, $(\mathsf{randBit}, \cdot)$, or $(\mathsf{divModConv}, \cdot)$ from $\mathcal{F}_{\mathsf{ABB}+\mathsf{conv}}$ the simulator emulates $\mathcal{F}_{\mathsf{ABB}}$ calling the functionality on behalf of virtual honest parties. The simulator only needs to know internal values of $\mathcal{F}_{\mathsf{ABB}}$ when they are opened, at which point it will receive the necessary values from $\mathcal{F}_{\mathsf{ABB}+\mathsf{conv}}$.
- Upon receiving command $(\mathsf{modConv}, 2^\ell, 2^k, \mathsf{id}_1, \mathsf{id}_2)$
  1. Emulate the calls of $\mathsf{randBit}$ to $\mathcal{F}_{\mathsf{ABB}}$ for $\mathsf{id}'_1, \ldots, \mathsf{id}'_\ell$ and $\mathsf{comb}$, resulting in $(r, \mathsf{id}'_0)$ and $(b, \mathsf{id}'_\ell)$.
  2. Then emulate the call $\mathsf{comb}$ to $\mathcal{F}_{\mathsf{ABB}}$ for $\mathsf{id}_1, \mathsf{id}'_0, \mathsf{id}'_\ell$ resulting in $(y, \mathsf{id}')$.
  3. Sample a random $y^* \leftarrow \mathbb{Z}_{2^\ell}$ and emulate the command $(\mathsf{open}, \mathsf{id}')$ by sending $y^*$ to the corrupt parties.
  4. Then emulate the final call $\mathsf{comb}$ to $\mathcal{F}_{\mathsf{ABB}}$ acting as the honest virtual parties.
- Upon receiving command $(\mathsf{modConv}, 2^\ell, 2^k, \mathsf{id}_1, \mathsf{id}_2, x)$ proceed as follows. (This is the case where conversion may fail as $x \geq 2^{\ell-1}$.) Since the simulator knows $x$, it runs the real protocol and instructs the ideal functionality to store the resulting output. In particular, the simulator does the following:

Fig. 8: Simulator for the Po2-Modulus-Conversion

Therefore the output of the protocol is correct. (2) Since the protocol correctly computes $x \mod 2^k$ it follows that the output does not depend on $b \in \{0, 1\}$ and $r \in [0, 2^{\ell-1})$. The mask $2^{\ell-1}b + r$ is uniform in $\mathbb{Z}_{2^\ell}$ for fresh uniform $r, b$, it follows the distribution of the opening $y$ is uniform and independent of $x$. Therefore the real opening $y$ is indistinguishable from $y^*$ picked by the simulator. □

## 4 Threshold FHE Decryption

### 4.1 Threshold BFV in MPC

We now describe our threshold decryption protocol ThBFV (Figure 9), which realizes the $\mathcal{F}_{\mathsf{ABB}+\mathsf{BFV}}$ (Figure 4) functionality in the $\mathcal{F}_{\mathsf{ABB}+\mathsf{Conv}}$ hybrid model.

We assume that the scaling factor $\Delta$ divides the ciphertext modulus $q$. This is a standard requirement in BFV-like schemes and can be ensured at the parameter selection stage, or by modulus switching. Moreover, we require that the error term bounded in $[0, \Delta - \Delta/\beta]$, shifting the error to be non-negative for the sake of convenience. Note that if the special purpose modulus conversion for powers of two is used, we can only support $\beta = 2$.

**High-level idea.** Let $[\Delta \cdot m + e]_q$ be the shared value obtained after the linear part of the decryption (we write $f_c([\mathsf{sk}]_q)$). Our goal is to recover $m$ while pre-

serving secrecy throughout the computation. As $\Delta$ divides $q$, we may directly convert,

$$[\Delta \cdot m + e]_q \to [\Delta \cdot m + e \bmod \Delta]_\Delta = [e]_\Delta.$$

Reducing the shared value modulo $\Delta$ yields a sharing of the error term $e$ modulo $\Delta$.

We then apply our modulus conversion protocol to lift this sharing back to modulus $q$, obtaining a sharing $[e]_q$. This correctly converts the modulus of $e$ whenever $e \in [0, \Delta - \Delta/\beta]$ which holds by assumption on our ciphertext. Once $[e]_q$ is obtained, we can subtract it from the initial value to isolate the message term:

$$[\Delta \cdot m]_q = [\Delta \cdot m + e]_q - [e]_q.$$

At this point, no sensitive information remains hidden in the sharing, and the value can be safely opened or post-processed.

**Protocol** $\mathsf{ThBFV}(q, t)$

Let $\Delta = q/t$, this protocol is in the $\mathcal{F}_{\mathsf{ABB+Conv}}(\{q, \Delta\})$ hybrid model, parameterized by domains $\mathbb{Z}_\Delta$ and $\mathbb{Z}_q$,

**I/O:** Given input $c$ and previously stored $[\mathsf{sk}]_q$ such that $f_c(\mathsf{sk}) = \Delta m + e$ with $e < \Delta - \Delta/\beta$, produces $[\Delta \cdot m]_q$ stored in $\mathcal{F}_{\mathsf{ABB+Conv}}$.

**Execute:** Each party $P_i$ proceeds as follows:

1. Use $\mathcal{F}_{\mathsf{ABB+Conv}}$ to compute $[\Delta \cdot m + e]_q \leftarrow f_c([\mathsf{sk}]_q)$
2. Call $\mathcal{F}_{\mathsf{ABB+Conv}}.\mathsf{divModConv}([\Delta \cdot m + e]_q) \to [e]_\Delta$.
3. Call $\mathcal{F}_{\mathsf{ABB+Conv}}.\mathsf{convert}([e]_\Delta) \to [e]_q$
4. Use $\mathcal{F}_{\mathsf{ABB+Conv}}$ to compute $[\Delta \cdot m]_q \leftarrow [\Delta \cdot m + e]_q - [e]_q$

Fig. 9: Protocol of threshold decryption of a BFV ciphertext.

We proceed to prove security.

**Theorem 4.** *The protocol* $\mathsf{ThBFV}(q, t)$ *(Figure 9) where* $\Delta = q/t$ *realizes the functionality* $\mathcal{F}_{\mathsf{ABB+BFV}}(\{q, \Delta\})$ *(Figure 2) in the* $\mathcal{F}_{\mathit{ABB+Conv}}(\{q, \Delta\})$*-hybrid model.*

*Proof.* We define a simulator for our protocol.

**Simulator $\mathcal{S}$**

As previously $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{ABB}}$ toward the adversary acting as the virtual honest parties.

- Upon receiving $((\mathsf{bfvDec}, q, t, c, \mathsf{id}_{\mathsf{sk}}, \mathsf{id}_{\mathsf{out}}))$ from $\mathcal{F}_{\mathsf{ABB+BFV}}$. (The case where conversion succeeds). The simulator proceeds acts as the honest parties invoking each command of $\mathcal{F}_{\mathsf{ABB+Conv}}$. In this case convert does not reveal anything to the adversary, so can be simulated without knowledge of the error $e$.
- When the simulator receives $((\mathsf{bfvDec}, q, t, c, \mathsf{id}_{\mathsf{sk}}, \mathsf{id}_{\mathsf{out}}), x)$ from $\mathcal{F}_{\mathsf{ABB+BFV}}$. (The case where conversion fails). Proceed through the operations of $\mathcal{F}_{\mathsf{ABB+Conv}}$ as specified in the protocol. At convert provide $e$ where $e = x \bmod \Delta$ to the adversary and wait for it to provide $e' \in \mathbb{Z}_q$, compute $x - e'$ and return this to the ideal functionality $\mathcal{F}_{\mathsf{ABB+BFV}}$.
- Forward all other commands to the emulated $\mathcal{F}_{\mathsf{ABB+Conv}}$ acting as the honest parties.

In the case where $f_c(\mathsf{sk}) = \Delta m + e$ with $e < \Delta - \Delta/\beta$. Here all operations of $\mathcal{F}_{\mathsf{ABB+Conv}}$ within the protocol are done correctly meaning that $[\Delta \cdot m]_q$ is computed correctly in the real world by construction, as is the case in the ideal world.

When $e \geq \Delta - \Delta/\beta$, the real adversary is allowed to control the conversion, using $x$ the adversary can ensure that the value returned to the ideal functionality is the same as what would be stored in the real world.

□

### 4.2 Threshold BGV in MPC

Next we describe our threshold decryption protocol ThBGV (Figure 10), which realizes the $\mathcal{F}_{\mathsf{ABB+BGV}}$ (Figure 3) functionality in the $\mathcal{F}_{\mathsf{ABB+Conv}}$ hybrid model.
**High-level idea.** The protocol closely follows the structure of the standard BGV decryption. Let

$$[m + t \cdot e]_q = f_c([\mathsf{sk}])$$

be the shared value obtained after the linear decryption step. Our goal is to recover $m$ while preserving the secrecy of intermediate values.

To achieve this, we directly apply the modulus conversion procedure to remove the error term. Whenever $m + t \cdot e < q/2$ the modulus conversion will be correct. Therefore, we obtain

$$[m + et]_q \to [m + et \bmod t]_t = [m]_t$$

**Theorem 5.** *The protocol* $\mathsf{ThBGV}(\mathsf{params}, \mathsf{sk}, c)$ *(Figure 10) realizes the functionality* $\mathcal{F}_{\mathsf{ABB+BGV}}(\{q, t\})$ *(Figure 2) in the* $\mathcal{F}_{\mathsf{ABB+Conv}}(\{q, t\})$*-hybrid model.*

**Protocol** $\mathsf{ThBGV}(q,t)$

This protocol is in the $\mathcal{F}_{\mathsf{ABB+Conv}}(\{q,t\})$ hybrid model, parameterized by domains $\mathbb{Z}_t$ and $\mathbb{Z}_q$.

**I/O:** Given input $c$ and previously stored $[\mathsf{sk}]_q$ such that the $f_c(\mathsf{sk}) < q - \frac{q}{\beta}$, produces $[m]_t$ stored in $\mathcal{F}_{\mathsf{ABB+Conv}}$.

**Execute:** Each party $P_i$ proceeds as follows:

1. Use $\mathcal{F}_{\mathsf{ABB+Conv}}$to compute $[m+et]_q \leftarrow f_c([\mathsf{sk}]_q)$
2. Call $\mathcal{F}_{\mathsf{ABB+Conv}}.\mathsf{convert}([m+et]_q) \rightarrow [m]_t$

Fig. 10: Protocol for threshold decryption of a BGV ciphertext.

*Proof.* For convenience, let $\mathcal{F}_{\mathsf{ABB+Conv}}$ be shorthand for $\mathcal{F}_{\mathsf{ABB+Conv}}(\{q,t\})$. We define a simulator $\mathcal{S}$ which emulates $\mathcal{F}_{\mathsf{ABB+Conv}}$ and the virtual honest parties.

**Simulator** $\mathcal{S}$

- Upon receiving command $(\mathsf{bgvDec}, q, t, c, \mathsf{id}_{sk}, \mathsf{id}_{out})$, proceed as follows.
  1. Emulate the call $\mathsf{comb}$ to $\mathcal{F}_{\mathsf{ABB+Conv}}$ for $\mathsf{id}_{sk}, c$ resulting in $(m + et, \mathsf{id}'_2)$.
  2. Emulate the calls of $\mathsf{conv}$ to $\mathcal{F}_{\mathsf{ABB+Conv}}$ on $(m + et, q, id_0)$ with the target modulus $t$, resulting in $(m, t, \mathsf{id}_{out})$ .
- Upon receiving command $((\mathsf{bgvDec}, q, t, c, \mathsf{id_{sk}}, \mathsf{id_{out}}), x)$ proceed as follows. (This is the case where conversion may fail as $x = f_c(\mathsf{sk}) \geq q - \frac{q}{\beta}$.) In the call to $\mathsf{convert}$ provide $x \in \mathbb{Z}_q$ to the adversary and wait for $y \in \mathbb{Z}_t$. Return $y$ to the ideal functionality $\mathcal{F}_{\mathsf{ABB+BGV}}$.
- Forward all other commands to the emulated $\mathcal{F}_{\mathsf{ABB+Conv}}$ acting as the honest parties.

We focus only on $(\mathsf{conv}, \cdot)$ as the remaining commands are simply forwarded to $\mathcal{F}_{\mathsf{ABB+Conv}}$ emulated internally in the simulator. When $x \geq q - \frac{q}{\beta}$ the real and ideal worlds are identical as the simulator given $x$ may allow the adversary to control the conversion in the emulated $\mathcal{F}_{\mathsf{ABB+Conv}}$ and store the resulting output as the decrypted message, exactly as in the real world.

This leaves the case where $x < q - \frac{q}{\beta}$, here the real protocol produces the correct value and by inspection, $x = f_c([\mathsf{sk}]_q) = [m + et]_q$. Emulating $\mathcal{F}_{\mathsf{ABB+Conv}}$ toward the adversary requires no knowledge of $x$ in this case, allowing this to be done perfectly, ensuring indistinguishability.

□

### 4.3 Performance comparison

In this section, we evaluate the performance of our protocols. We focus on the decryption of correctly encrypted messages, assuming a noise bound fixed at setup that enables the use of our modulus conversion protocols for $\beta = 2$. This requires a BGV error bound of the form $|e_{\mathrm{BGV}}| < q/4t$ and a BFV error bound satisfying $|e_{\mathrm{BFV}}| < \Delta/4$. This choice of $\beta$ poses few restrictions on FHE parameter sets while keeping the preprocessing small. We note that other scenarios may require larger values of $\beta$ in which case the resulting protocol would require $2\beta - 1$ preprocessed values.

For the remainder of this section we let $k = \lceil \log q \rceil$ be the bit-length of the ciphertext modulus, and $l = \lceil \log t \rceil$ be the bit-length of the plaintext modulus. Note, the bit length for an intermediary modulus $Q$ introduced by general modulus conversion is $\sigma$ bits longer than the origin modulus, where $\sigma$ is the statistical security parameter. For BFV, we use the modulus $\Delta = q/t$, which is of bit-length $n = k - l = \lceil \log \Delta \rceil$.

**BGV.** BGV is typically instantiated with a ciphertext modulus and the plaintext modulus that are co-prime. This however, does not require the plaintext modulus $t$ to be prime. When $t$ is prime it is more efficient to use our prime conversion protocol (Figure 5), rather than the indirect general conversion (Figure 6).

*When $t$ is prime.* Here we may realize the conversion functionality with our Prime-Modulus-Conversion, protocol. In this case, the protocol opens two values in $\mathbb{Z}_q$ in the first round, one value in $\mathbb{Z}_t$ in the second round for the polynomial evaluation, and one value in $\mathbb{Z}_t$ in the round of reconstruction. This results in a total communication cost per party of $2k + 2l$ bits over three opening rounds.
Regarding preprocessing, each party requires two shared random values in $\mathbb{Z}_q$ (for masking and opening), along with their shared representation in $\mathbb{Z}_t$, as well as three shared random values in $\mathbb{Z}_t$ for polynomial evaluation. This result in a total of $2k + 5l$ bits of preprocessed data per party.

*When $t$ is not prime.* Here we instead use General-Modulus-Conversion. The protocol first calls Prime-Modulus-Conversion, which opens two values in $\mathbb{Z}_q$ and one value in $\mathbb{Z}_Q$ in 2 round. Then open one more masked value in $\mathbb{Z}_Q$ and finally reconstruct opening in $\mathbb{Z}_t$ in two more rounds. The resulting communication cost per party is $2k + 2(k + \kappa) + l$ bits over 4 round of opening. However, using the optimization discussed in the modulus conversion section, we can perform the two last openings at the same time when comes the time to reconstruct the message leading to a 3 opening round protocol.
For preprocessing, each party uses two shared random values in $\mathbb{Z}_q$, along with their representation in $\mathbb{Z}_Q$, three shared random values in $\mathbb{Z}_Q$ for polynomial evaluation and one shared random value in $\mathbb{Z}_Q$ together with its representation in $\mathbb{Z}_t$ for the final opening. This yields a total of $2k + 6(k + \kappa) + l$ bits of preprocessed data per party.

**BGV.** In contrast to BGV, our BFV protocol is instantiated with a call to the conversion functionality from $\Delta$ to the ciphertext modulus $q$. Here $\Delta$ is assumed

to divide the ciphertext modulus. We split our analysis into two cases, the general case, assuming $\Delta | q$ and the special case where $\Delta$ and $q$ are powers-of-two.

*Powers-of-two.* When $\Delta$ and $q$ are power of 2 we realize the convert functionality with the Po2-Modulus-Conversion protocol. In this case, the protocol opens one values in $\mathbb{Z}_\Delta$ in the first round and one value in $\mathbb{Z}_q$ when reconstructing the message. This results in a total communication cost per party of $2k - l$ bits over two round.
For preprocessing each party requires two shared random values, one in $\mathbb{Z}_{2^{k-l-1}}$ and one bit (for masking and opening), their shared representation in $\mathbb{Z}_\Delta$ can be locally constructed so doesn't contribute additional stored values. This leads to a total of $k - l$ bits of preprocessed data per party.

*General $\Delta$ divides $q$.* In the general case, we instead use our general modulus conversion protocol. We end up opening two values in $\mathbb{Z}_\Delta$, four values in $\mathbb{Z}_Q$ and one value in $\mathbb{Z}_t$. The resulting communication cost per party is $2n + 4(n + \kappa) + k$ bits over four opening rounds.
For preprocessing, each party uses two shared random values in $\mathbb{Z}_\Delta$, along with their representation in $\mathbb{Z}_Q$, three shared random values in $\mathbb{Z}_Q$ for polynomial evaluation and one shared random value in $\mathbb{Z}_Q$ together with its representation in $\mathbb{Z}_q$ for the final opening. This yields a total of $2n + 6(k + \kappa) + k$ bits of preprocessed data per party.

We previously summarized these results in Table 1.

**Acknowledgements.** This research was supported by: the European Research Council (ERC) under the European Unions's Horizon 2020 research and innovation programme under grant agreement No 101124977 (DECRYPSYS); the Next-Generation Cybersecurity programme (Digital Research Centre Denmark (DIREC), National Defence Technology Centre (NFC) and Danish Industry Foundation under project 2025-0766).

## References

AOR+19. Abdelrahaman Aly, Emmanuela Orsini, Dragos Rotaru, Nigel P Smart, and Tim Wood. Zaphod: Efficiently combining lsss and garbled circuits in scale. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 33–44, 2019.

BCD+25. Carl Bootland, Kelong Cong, Daniel Demmler, Tore Kasper Frederiksen, Benoit Libert, Jean-Baptiste Orfila, Dragos Rotaru, Nigel P. Smart, Titouan Tanguy, Samuel Tap, and Michael Walter. Threshold (fully) homomorphic encryption. Cryptology ePrint Archive, Report 2025/699, 2025.

BD10. Rikke Bendlin and Ivan Damgård. Threshold decryption and zero-knowledge proofs for lattice-based cryptosystems. In Daniele Micciancio, editor, *TCC 2010*, volume 5978 of *LNCS*, pages 201–218, Zurich, Switzerland, February 9–11, 2010. Springer Berlin Heidelberg, Germany.

BdCE+25. Alexander Bienstock, Leo de Castro, Daniel Escudero, Antigoni Polychroniadou, and Akira Takahashi. Quorus: Efficient, scalable threshold ML-DSA signatures from MPC. Cryptology ePrint Archive, Paper 2025/1163, 2025.

BGG+17. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. Cryptology ePrint Archive, Paper 2017/956, 2017.

BGV12. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012*, pages 309–325, Cambridge, MA, USA, January 8–10, 2012. ACM.

Bra12. Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 868–886, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany.

BS23. Katharina Boudgoust and Peter Scholl. Simple threshold (fully homomorphic) encryption from LWE with polynomial modulus. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part I*, volume 14438 of *LNCS*, pages 371–404, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.

CDE+18. Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. SPD $\mathbb{Z}_{2^k}$: Efficient MPC mod $2^k$ for dishonest majority. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 769–798, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.

CDI05. Ronald Cramer, Ivan Damgård, and Yuval Ishai. Share conversion, pseudorandom secret-sharing and applications to secure computation. In Joe Kilian, editor, *TCC 2005*, volume 3378 of *LNCS*, pages 342–362, Cambridge, MA, USA, February 10–12, 2005. Springer Berlin Heidelberg, Germany.

CGGI20. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, January 2020.

DEF+19. Ivan Damgård, Daniel Escudero, Tore Kasper Frederiksen, Marcel Keller, Peter Scholl, and Nikolaj Volgushev. New primitives for actively-secure MPC over rings with applications to private machine learning. In *2019 IEEE Symposium on Security and Privacy*, pages 1102–1120, San Francisco, CA, USA, May 19–23, 2019. IEEE Computer Society Press.

DFK+06. Ivan Damgård, Matthias Fitzi, Eike Kiltz, Jesper Buus Nielsen, and Tomas Toft. Unconditionally secure constant-rounds multi-party computation for equality, comparison, bits and exponentiation. In Shai Halevi and Tal Rabin, editors, *TCC 2006*, volume 3876 of *LNCS*, pages 285–304, New York, NY, USA, March 4–7, 2006. Springer Berlin Heidelberg, Germany.

DOK+20. Anders P. K. Dalskov, Claudio Orlandi, Marcel Keller, Kris Shrishak, and Haya Shulman. Securing DNSSEC keys via threshold ECDSA from generic MPC. In Liqun Chen, Ninghui Li, Kaitai Liang, and Steve A. Schneider, editors, *ESORICS 2020, Part II*, volume 12309 of *LNCS*, pages 654–673, Guildford, UK, September 14–18, 2020. Springer, Cham, Switzerland.

EGK+20. Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 823–852, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.

Esc22. Daniel Escudero. An introduction to secret-sharing-based secure multiparty computation. Cryptology ePrint Archive, Report 2022/062, 2022.

FV12. Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012.

HLW+25. Zhenkai Hu, Haofei Liang, Xiao Wang, Xiang Xie, Kang Yang, Yu Yu, and Wenhao Zhang. Ajax: Fast threshold fully homomorphic encryption without noise flooding. Cryptology ePrint Archive, Paper 2025/1834, 2025.

OSV20. Emmanuela Orsini, Nigel P. Smart, and Frederik Vercauteren. Overdrive2k: Efficient secure MPC over $\mathbb{Z}_{2^k}$ from somewhat homomorphic encryption. In Stanislaw Jarecki, editor, *CT-RSA 2020*, volume 12006 of *LNCS*, pages 254–283, San Francisco, CA, USA, February 24–28, 2020. Springer, Cham, Switzerland.

OT25. Hiroki Okada and Tsuyoshi Takagi. Low communication threshold FHE from standard (module-)LWE. Cryptology ePrint Archive, Paper 2025/409, 2025.

PS24. Alain Passelègue and Damien Stehlé. Low communication threshold fully homomorphic encryption. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 297–329, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.

RW19. Dragos Rotaru and Tim Wood. MArBled circuits: Mixing arithmetic and Boolean circuits with active security. In Feng Hao, Sushmita Ruj, and Sourav Sen Gupta, editors, *INDOCRYPT 2019*, volume 11898 of *LNCS*, pages 227–249, Hyderabad, India, December 15–18, 2019. Springer, Cham, Switzerland.

ZZLP25. Guy Zyskind, Doron Zarchy, Max Leibovich, and Chris Peikert. High-throughput universally composable threshold FHE decryption. Cryptology ePrint Archive, Paper 2025/1781, 2025.

# A Sub-Protocols and Preprocessing

## A.1 Subprotocol

The polynomial evaluation procedure we will use is implemented using a standard mask-and-open approach. The parties first mask the shared input with a random shared value and open the masked value. Then evaluate the polynomial on the masked value minus the mask's shares. To do this evaluation requires holding every power of the mask's shares so a preprocessing of $2\beta - 1$ values that we describe below. We describe the protocol explicitly in Figure 11 and refer to it as a subroutine throughout the the paper.

## A.2 Preprocessing

For the concrete instantiation of our protocols Prime-Modulus-Conversion, General-Modulus-Conversion, Po2-Modulus-Conversion, ThBGV, and ThBFV, we require a number of preprocessing procedures.

Both ThBGV and ThBFV rely on the General-Modulus-Conversion and Prime-Modulus-Conversion protocol, which themselves invokes a subroutine PolyEval. This subroutine evaluates a polynomial of degree $2\beta - 1$, and therefore requires preprocessed sharings of the form $[y]$, $[y^2], \ldots [y^{2\beta-1}]$.

**Protocol** $\mathsf{PolyEval}([x]_q, P(X)) \to [P(x)]_q$

This protocol is in the $\mathcal{F}_{\mathsf{ABB}}(q)$ hybrid model, parameterized by domain $\mathbb{Z}_q$, We assume $q$ is prime and $\beta = \deg(P)$.

**Preprocessing:** $\{[y^i]_q\}_{i\in[\beta]}$, where $y \in \mathbb{Z}_q$ is uniformly random.
**I/O:** Given input $[x]_q$ and $P(X)$, produces $[P(x)]_q$ stored in $\mathcal{F}_{\mathsf{ABB}}$.
**Execute:**

1. Use $\mathcal{F}_{\mathsf{ABB}}$ to compute $[z]_q = [x]_q + [y]_q$
2. Call $\mathcal{F}_{\mathsf{ABB}}.\mathsf{Open}([z]_q) \to z$.
3. Use $\mathcal{F}_{\mathsf{ABB}}$ to compute $[P(x)]_q \leftarrow P(z - [y]_q)$

Fig. 11: Protocol to evaluate a secret shared values on any polynomial.

In addition, $\mathsf{General\text{-}Modulus\text{-}Conversion}(\mathbb{Z}_q, \mathbb{Z}_t, .)$ and $\mathsf{Prime\text{-}Modulus\text{-}Conversion}(\mathbb{Z}_q, \mathbb{Z}_t, .)$requires preprocessed randomness in the form of shared values across different moduli. More precisely, we assume access to sharings of the form $[r]_q$ and $[r]_Q$ or $[r]_q$ and $[r]_t$, where $r$ is sampled in $\mathbb{Z}_q$ uniformly at random, Q the intermediary prime modulus of the subprotocole and $t$ the plaintext modulus. These correlated sharings are used to securely mask values during the conversion process.

**Preprocessing of powers of $y$** This task can be done using standard methods, using only the interface of the ABB. The simplest solution is to let all players input a random number modulo $Q$ to the ABB, add them securely to get $[y]_Q$, and finally do $2\beta - 2$ multiplications to get the required powers of $y$.

**Preprocessing of $[r]_q$ and $[r]_Q$** We now describe how to generate correlated sharings $[r]_q$ and $[r]_Q$ of a value $r$ chosen uniformly from $\mathbb{Z}_q$.

One might think that, at least for semi-honest security, we could ask each party $P_i$ to input $[r_i]_q, [r_i]_Q$ for a random $r_i$, and just add these pairs over all players, so we output $\sum_i [r_i]_q, \sum_i [r_i]_Q$. But this does not work in general, if the sum creates overflow modulo both $q$ and $Q$, we will not get the same number modulo $q$ and $Q$.

So instead, we can first observe that it is sufficient to generate *double-shared bits*, pairs of form $[b]_q, [b]_Q$ for a random $0/1$ value $b$. Now, in the most common case, where $q = 2^k$, we can first generate $k$ pairs $([b_j]_q, [b_j]_Q), j = 0 \dots k-1$ and then we can set

$$[r]_q = \sum_{j=0}^{k-1} 2^j [b_j]_q, \;\; [r]_Q = \sum_{j=0}^{k-1} 2^j [b_j]_Q$$

Clearly, the first ABB variable will contain $r$ chosen uniformly chosen modulo $q$, and the second will contain $r$ mod $Q$, as desired.

If $q$ is not a 2-power, we choose $k$ such that $2^{k-1} < q < 2^k$ and do exactly what we just described to get $[r]_q, [r]_Q$. Note that, conditioned on the event that $r < q$, r is uniform in $\mathbb{Z}_q$. So we then compute securely a binary circuit that decides if $r < q$ and output the result. Since $q$ is public and we have the bits of $r$ in the ABB, this is straightforward and will cost $O(k)$ secure multiplications. If indeed $r < q$, we keep the pair, otherwise we discard and try again. Note that we will succeed with probability at least $1/2$.

We also may need to compute pairs of form $[z]_Q, [z]_t$, but here the only requirement is that $z$ is a random $v$-bit number where $2^v$ is maximal such that $2^v < Q$. This is also straightforward to obtain from double-shared bits in much the same way as above.

We generate $([z_j]_Q, [z_j]_t), j = 0 \dots v-1$ and then we can set

$$[z]_Q = \sum_{j=0}^{v-1} 2^j [z_j]_q, \;\; [z]_Q = \sum_{j=0}^{v-1} 2^j [z_j]_Q$$

**Double-shared bits from da-bits.** Da-bits are pairs of form $[b]_Q, [b]_2$, where $b$ is random bit and $[b]_2$ refers to an ABB variable defined over $\mathbb{Z}_2$. They have been extensively studied, see [AOR+19], [EGK+20], for instance.

Given two da-bits of form $[b]_Q, [b]_2, [b']_q, [b']_2$, it is trivial to generate a double-shared bit, we compute and open $b \oplus b'$. This is a random bit that is easy to simulate. If this value is 0, we output $[b]_Q, [b']_q$, else we output $[b]_Q, [1 - b']_q$. In [AOR+19] an efficient protocol for da-bits is shown for the case $[b]_Q, [b]_2$ where $Q$ is prime, but there are also protocols that are black-box in the domain. Several existing protocols are for the dishonest majority case.

**Double-shared bits, Honest super-majority.** In this subsection, we consider the case where we have less than $1/3$ corruptions, i.e. $n > 3t$. We also concentrate on small values of $n$ which is the most relevant case when doing threshold cryptography.

To generate double-shared bits in this setting, we "open the ABB box" and consider concrete forms of secret sharing. So in this subsection, we abuse the notation and let, e.g., $[x]_Q$ stand for Shamir secret sharing of $x$ modulo $Q$.

If $q$ and $t$ are prime we will also use Shamir, but otherwise notation like $[x]_q$ will denote replicated sharing of $x$, which is feasible to use for small $n$ Replicated sharing of $x$ means the following: for each player-subset $A$ of size $n-t$, each player in $A$ holds an integer $x_A \in \mathbb{Z}_q$, such that $x = \sum_A x_A \bmod q$.

The notation $[u]_{\mathbb{Z}}$ will denote a replicated *integer* secret sharing of the number $u$. This means the following: for each player-subset $A$ of size $n - t$, each player in $A$ holds an integer $u_A$, and we have $u = \sum_A u_A$.

Sharings of form $[u]_{\mathbb{Z}}$ can be generated locally with computational security, under a set-up assumption where each player in each subset $A$ is given a key $K_A$ for a PRF $\phi$. Then each player can locally generate $u_A = \phi_K(a)$ where $a$ is some publicly known nonce. We will assume in the following that the random $u_A$'s are all chosen to be of length $\kappa$ bits, where $\kappa$ is a statistical security parameter.

Note also that, by standard techniques, $[u]_{\mathbb{Z}}$ can be locally converted to a Shamir sharing $[u]_Q$ modulo $Q$. Namely, each player will reduce the shares they know modulo $Q$ and apply the share conversion technique from [CDI05].

For the following protocol to work, we will assume that the prime $Q$ is of bit length $\kappa + n$. Note that this means that if we convert $[u]_{\mathbb{Z}}$ to $[u]_Q$, we will get a sharing of the integer $u$ and not $u$ mod $Q$. This is because the sum $u = \sum_A u_A$ is less than $Q$ since there are at most $2^n$ subsets $A$.

Finally, note that using a standard technique from [DFK+06], one can generate $[b]_Q$ where $b$ is a random bit, unknown to all players. This costs two rounds (one secure multiplication and one opening).

The protocol $\pi_{pair}$ following from these obervations is in Figure 12.

**Protocol** $\pi_{pair} \rightarrow [b]_q, [b]_Q$

This protcool works for a malicious adversary corrupting $t < n/3$ players.

1. Generate $[b]_Q$ and $[u]_{\mathbb{Z}}$.
2. Convert $[u]_{\mathbb{Z}}$ to $[u]_Q$ and $[u]_q$
3. Open $[u]_Q + [b]_Q$ to get $v = u + b$.
4. Compute $v - [u]_q = [b]_q$

Fig. 12: Protocol for generating secret sharings of the same bit in different domains.

$\pi_{pair}$ is correct: first note that since $n > 3t$, Shamir sharings can be opened by simply sending all shares to all players who can recover the secret via error correction. Further, by the observations above, $v$ will indeed be the integer sum of $u$ and $b$ with no overflow modulo $Q$. Therefore we also have $v = (u+b) \bmod q$, which shows that the final step indeed generates $[b]_q$,m as desired.

$\pi_{pair}$ is statistically secure. To see this, note that the only published data is $v$, which can be written as $v = \sum_A u_A + b$. At least one of the $u_A$ will be unknown to the adversary, and clearly $u_A + b$ is statistically indistinguishable from a random $\kappa + n$-bit number. So $v$ is easy to simulate.

Finally, it is easy to see that $\pi_{pair}$ runs in 3 rounds.

**Double-shared bits, Malicious, Honest majority** We note that for honest majority, $n > 2t$, we can use the exact same protocol as for $n > 3t$. We will now only get security with abort. Namely, we run $\pi_{pair}$, but each time a Shamir-sharing is opened, we require that all shares be on the same polynomial, or else we abort. This ensures that the protocol continues with the correct secret or aborts, and this is all that is needed for the same proof as above to go through.

**Double-shared bits, Semi-honest, Full threshold** For this setting, we can once again reuse $\pi_{pair}$, now assuming a semi-honest adversary corrupting $t = n-1$ players. In this case, replicated secret-sharing reduces to additive sharing, so we do not need to assume $n$ is small. Also, there is no issue of players lying about their shares when opening. The only caveat is that the secure multiplications will now require cryptographic machinery, such as OT, but it can nevertheless be done very efficiently in constant round, using OT extension, for instance.

**Double-shared bits, Generic protocol** We can make double-shared bits in a generic way using only the ABB interface. This is not a particularly efficient solution, but it shows we can do the preprocessing for any implementation of the basic ABB, including the hardest case of dishonest majority and malicious adversary, by using known ABB protocols, such as SPDZ or variants thereof.

The first observation is that if we assume a protocol $\pi_{pair}^{verify}$ that a player $P_i$ can use to input a bit $b_i$ modulo both $q$ and $Q$: $[b_i]_q, [b_i]_Q$ and prove this was done correctly, we can get the desired result by setting

$$[b]_q = \oplus_{i=1}^n [b_i]_q,\ [b]_Q = \oplus_{i=1}^n [b_i]_Q,$$

where the XOR operation must be implemented via aritmetic modulo $q$ ($Q$). Here we can use the fact that $b \oplus b' = b + b' - 2bb' \bmod q$. This will require $\log n$ rounds when implemented in the straightforward way.

The idea for constructing $\pi_{pair}^{verify}$ is as follows: the player creating the pair will be called the dealer $D$. First, $D$ inputs $[b]_q, [b]_Q$. Then, to show that the two values are bits and are equal, we use a variant of a so-called $\Sigma$-protocol. Namely, we repeat the following security parameter number of times: a new auxiliary pair of bits $[c]_q, [c]_Q$ is created, and then a random bit $e$ is created and opened. If $e = 0$, both sharings $[c]_q, [c]_Q$ are opened and we check that the values are bits and are equal. If $e = 1$, $[b \oplus c]_q, [b \oplus c]_Q$ are computed and opened, and again it is verified that the values are bits and are equal.

If $D$ is honest, it is easy to see that the adversary's view can be simulated: we will always see the same uniformly random bit be opened modulo both $q$ and $Q$.

For corrupt $D$ things are more complicated, since the values input need not be binary, and we need to show that there is no attack where $D$ can get away with non-binary input. Modulo $Q$ (which is a prime) this is easy to solve. Instead of letting $D$ choose values, we can use the efficent method from [DFK+06] to generate a random shared bit and open it to (only) $D$. As it turns out, this is sufficient to force the values modulo $q$ to also be binary. The resulting protocol is in Figure 13.

An honest $D$ will clearly always succeed, and we can easily simulate the protocol, as observed above.

For corrupt $D$, assume $D$ inputs arbitrary values modulo $q$: $[b']_q, [c'_1]_q, \ldots [c'_\kappa]_q$. We want to show that if $b' \neq b$, $D$ is disqualified with overwhelming probability.

**Protocol $\pi_{pair}^{\text{verify}}$params**

This protocol works in the ABB hybrid model, for a malicious adversary corrupting any number of players.

1. Generate $[b]_Q$ and open it towards $D$.
2. $D$ inputs $[b]_q$.
3. Generate shared bits $[c_1]_Q, \dots [c_\kappa]_Q$ and $[e_1], \dots [e_\kappa]_Q$.
4. Open $c_1, \dots c_\kappa$ towards $D$.
5. $D$ inputs $[c_1]_q, \dots [c_\kappa]_q$.
6. Open $e_1, \dots e_\kappa$.
7. For each $j = 1, \dots \kappa$:
   - if $e_j = 0$, open $[c_j]_Q, [c_j]_q$ to get $c_j^q, c_j^Q$ and verify that $c_j^q = c_j^Q$ (note that $c_j^Q$ is guaranteed to be 0 or 1).
   - if $e_j = 1$, compute and open $[b \oplus c_j]_Q, [b \oplus c_j]_q$ to get $d_j^Q, d_j^q$ and verify that $d_j^Q = d_j^q$.
8. if any check fails, $D$ is disqualified and his input values are ignored in any protocol calling this one.

Fig. 13: Protocol for a dealer $D$ to generate secret sharings of the same bit in different domains.

Assume for contradiction that $D$ has $b \neq b'$ and can survive with non-negligible probability. Note that all the values chosen by $D$ are fixed before the $e_j$'s are opened.

If these values violate the condition checked in the protocol when $e_j = 0$, for more than half the values of $j$, $D$ is disqualified with overwhelming probability. The same clearly holds for the condition checked when $e_j = 1$.

It follows that there must be at least one value of $j$ for which both conditions are satisfied, in other words that $c_j = c'_j$ and $b + c_j - 2bc_j = b' + c'_j - 2bc'_j \bmod q$. These together imply the equation

$$b - b' = 2c_j(b - b') \bmod q.$$

Recall that $c_j$ is always 0 or 1, and in both cases we conclude that $b - b' = 0$ which is a contradiction.